A type-preserving compiler from a systems language to a dependently typed assembly language

9615Q

2025-2026

Declaration of Originality

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 9615Q, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Signed *Athena Eng*
Date *April 20, 2026*

Proforma

Candidate Number	9615Q
Title	A type-preserving compiler from a systems language to DTAL
Examination	Computer Science Tripos – Part II, 2026
Word Count	
Code Line Count	
Project Originator	The candidate
Project Supervisor	Yulong Huang
Marking Supervisor	Jeremy Yallop

Original aims of the project

The project set out to develop a type-preserving compilation toolchain for a systems programming language with refinement types. The goal was to demonstrate that structural safety properties, such as spatial memory safety and control-flow integrity, could be mathematically guaranteed in low-level executable code without trusting the complex compiler itself. This involved designing a source language, building a compiler that lowers high-level constructs to a Dependently Typed Assembly Language (DTAL) while preserving type information, and creating an independent verifier capable of confirming the safety of the resulting DTAL independently of the compiler.

Work completed

Special difficulties

None.

Contents

In his seminal 1977 paper, Leslie Lamport defined *safety* as a property ensuring that "something bad never happens" [1]. In modern systems programming, where performance-critical code often runs with administrative privileges, *memory safety* translates to a guarantee: no out-of-bounds reads, no null-pointer dereferences, and no undefined behaviour. Yet, safety-critical systems often rely on systems languages like C and C++ for performance and low-level control, accepting their inherent unsafety. The history of software is littered with fatal safety violations. The Heartbleed bug [2] and the Therac-25 accidents [3] were not failures of logic, but failures of structural safety such as an unchecked index or a race condition.

Current solutions exist at two extremes. At one end are verified compilers like CompCert [4], which provide foundational proofs of correctness but are difficult to extend.

At the other are type-safe languages like Rust with language features such as ownership and borrow-checking to enforce compile-time memory safety, which provide developer productivity but require trusting an enormous and evolving compiler.

Veritas explores a third path: Proof-Carrying Code [5]. In the Veritas model, the compiler is untrusted. Safety is instead guaranteed by a standalone verifier that re-proves safety properties using only the type annotations in the assembly.

I Introduction

In his seminal 1977 paper, Leslie Lamport defined *safety* as a property ensuring that "something bad never happens" [1]. In modern systems programming, where performance-critical code often runs with administrative privileges, *memory safety* translates to a guarantee: no out-of-bounds reads, no null-pointer dereferences, and no undefined behaviour. Yet, safety-critical systems often rely on systems languages like C and C++ for performance and low-level control, accepting their inherent unsafety. Languages such as Rust enforce compile-time safety using language features such as ownership and borrow-checking to prevent undefined behaviour, but this requires trusting that the compiler implementation is correct.

This dissertation presents **Veritas**: a toolchain that attempts to address the compromise between performance and safety by moving the root of trust from a complex compiler to a small, independent verifier. By employing type-preserving compilation, Veritas carries mathematical proofs of safety from the high-level source to a target low-level representation.

1.1 Motivation: The Location of Trust

The history of software is littered with fatal safety violations. The Heartbleed bug [2] and the Therac-25 accidents [3] were not failures of logic, but failures of structural safety such as an unchecked index or a race condition.

Current solutions exist at two extremes. At one end are verified compilers like CompCert [4], which provide foundational proofs of correctness but are difficult to extend. At the other are type-safe languages like Rust, which provide developer productivity but require trusting an enormous and evolving compiler. Veritas explores a third path: Proof-Carrying Code [5]. In the Veritas model, the compiler is untrusted. Safety is instead guaranteed by a standalone verifier that re-proves safety properties using only the type annotations in the assembly.

1.2 Technical Approach: Refinement Types and Satisfiability Modulo Theories (SMT)

Veritas verifies safety properties through refinement types [6], which are types decorated with logical predicates. For example, the length of an array is not just an `int`, but a value of type $\{v : \texttt{int} \mid v \geq 0\}$. The programmer provides specifications—type annotations, function contracts, loop invariants—and the compiler automatically discharges the resulting proof obligations using a Satisfiability Modulo Theories (SMT) solver. Veritas also introduces a refinement synthesis engine; when the compiler encounters arithmetic (e.g., `i + 1`), it automatically synthesises a symbolic equality refinement ($\{v : \texttt{int} \mid v = i + 1\}$).

These refinements are discharged by the Z3 SMT solver [7], bridging the semantic gap between high-level contracts and low-level instructions. They can also be combined with known assumptions to derive further properties (see TODO). Crucially, Veritas prioritises structural safety over full semantic correctness; a program may produce an incorrect result, but it is guaranteed that its memory boundaries are never violated.

1.3 Aims and Contributions

The aim of this project is to deliver a type-preserving compilation pipeline that supports independent verification of systems code. The key contributions are:

- **The Veritas Language:** A systems language featuring refinement types, loop invariants, and function contracts.
- **Type-Preserving Backend:** A pipeline that lowers high-level types to a typed SSA (Static Single Assignment) intermediate representation, preserving all defined safety invariants.
- **x86 runtime generation:** An assembler that maps DTAL instructions to x86 and generates an executable ELF.
- **Dependently Typed Assembly (DTAL):** A target language based on Xi and Harper [8] that embeds type states and constraints directly into the instruction stream.
- **Independent Verifier:** A tool that re-verifies DTAL programs without access to the source code, reducing the trusted computing base to the verifier and the SMT solver.
- **Formal Evaluation:** A suite of programs used to measure the overhead and expressiveness of the architecture.

1.4 Dissertation Roadmap

Chapter 2 establishes the theoretical foundations, surveys related work, and describes the key design decisions. Chapter 3 describes the implementation of the compiler and verifier. Chapter 4 provides a quantitative evaluation of the toolchain's performance and safety guarantees. Finally, Chapter 5 reflects on the project's success and future directions.

2 Preparation

This chapter establishes the theoretical foundations of the project, surveys related work, and describes the design decisions that were made before implementation began.

2.1 Background

2.1.1 Type systems and safety

The fundamental theorem that connects type systems to safety is *type soundness*: a well-typed program cannot 'go wrong' at runtime [9]. Wright and Felleisen formalised this via the *progress* and *preservation* lemmas [10]:

Type systems exist on a spectrum of expressiveness. At one end, simply typed languages (such as C's type system) distinguish broad categories like integers, floats, and pointers. This catches type mismatches—such as passing a pointer where an integer is expected—but cannot constrain the values that inhabit a type. Hindley–Milner type systems [9], used in ML and Haskell, add parametric polymorphism and principal type inference, enabling the type checker to infer the most general type for every expression without annotations. However, these systems cannot express properties that depend on values; for instance, that an array index is within bounds, or that a division's denominator is non-zero. These properties lend themselves to safety-critical software, and expressing them requires types that can refer to values: *dependent types*.

Veritas targets this gap: its type system extends a simple base type system (integers, booleans, arrays) with refinement types, a decidable subset of dependent types, that encode value-dependent invariants.

2.1.2 Refinement types

In a dependently typed language, types may depend on terms. The canonical example is the type of vectors indexed by their length, $\mathrm{Vec}(A, n)$, where n is a natural number. A function that accesses the i-th element can require in its type that $0 \leq i < n$, statically ruling out out-of-bounds accesses. Full dependent type systems, such as those in Coq and Agda, can express arbitrary propositions as types (via the Curry–Howard correspondence), but this generality comes at a cost: type checking may require the programmer to construct explicit proof terms, and type inference is generally undecidable.

Refinement types [6] are a practical restriction of full dependent types in which a base type is decorated with a logical predicate drawn from a decidable logic. For example, the type $\{v : \texttt{int} \mid v > 0\}$ classifies integers that are strictly positive. Notably, subtyping between refinement types reduces to logical implication: $\{v : \texttt{int} \mid P(v)\} <: \{v : \texttt{int} \mid Q(v)\}$ holds when $P(v) \Rightarrow Q(v)$ is valid. Because the predicates are drawn from a decidable theory (typically quantifier-free linear arithmetic), this implication check can be delegated to an SMT solver.

A particularly useful special case is the singleton type, written $\texttt{int}(n)$, which is inhabited by exactly the integer n. Singleton types arise naturally from Xi and Pfenning's work on Dependent ML [11], where integer index expressions are used to track array sizes and loop bounds. In Veritas, singleton types propagate exact values through arithmetic: if $x : \texttt{int}(3)$ and $y : \texttt{int}(5)$, then $x + y : \texttt{int}(8)$. This enables compile-time verification of array bounds when indices are statically known. Singleton types can be viewed as the special refinement $\{v : \texttt{int} \mid v = n\}$; in the implementation, they are represented as a distinct type variant for efficiency, as constant folding on singletons avoids invoking the SMT solver entirely.

Veritas additionally supports bounded quantifiers in specification positions. A postcondition such as **forall** i **in** $0..n$ { $arr[i] \geq 0$ } expresses a property over all elements of an array, going beyond scalar refinements to reason about aggregate data structures.

2.1.3 Bidirectional type checking

Bidirectional type checking [12], structures a type system into two mutually recursive judgements: *checking* and *synthesis*:

- *Type checking* takes a context Γ, program e and type τ as input and produces a true/false output reflecting whether e is well-typed: $\Gamma \vdash e \leftarrow \tau$
- *Type synthesis* takes a context Γ and program e as input and produces a types τ as output: $\Gamma \vdash e \Rightarrow \tau$

The two modes are connected by a subsumption rule:

$$\frac{\Gamma \vdash e \Rightarrow \sigma \quad \sigma <: \tau}{\Gamma \vdash e \leftarrow \tau}$$

For refinement types, the bidirectional structure is particularly well-suited. Synthesis produces the most precise type available from the immediate structure of an expression—for integer literals, this is a singleton type; for arithmetic, it is a refinement capturing the exact relationship between the result and its operands. Checking then verifies subtyping against the expected type. The programmer only needs to annotate types at function boundaries and mutable variable declarations; the type checker infers intermediate refinement types automatically. Bidirectional type-checking offers a middle ground between Hindley-Milner full type inference and the need for complete type annotations on every expression.

In Veritas, the bidirectional structure also threads the typing context through the judgements. Both synthesis and checking accept a context and return an updated context, reflecting changes from mutable variable assignments and newly learned refinement propositions.

2.1.4 SMT-based verification

Satisfiability Modulo Theories are an extension of boolean satisfiability (SAT) to richer logics. An SMT solver decides the satisfiability of first-order formulae over a combination of background theories. The theories relevant to program verification include linear integer arithmetic (LIA), which supports addition, subtraction, and comparison of integers; the theory of arrays, which models arrays as functions from indices to values with `select` and `store` operations; and uninterpreted functions, which allow reasoning about function applications without knowing their implementation. Z3 [7], developed by Microsoft Research, is one of the most widely used SMT solvers and is the solver used in this project. It was chosen for its maturity and reliability, having won numerous SMT-COMP competitions [13]. Z3 is also precedent in other similar verification systems that have influenced Veritas' design such as Liquid Haskell, Dafny, and F*.

The technique for using an SMT solver as a refinement type checker is *proof by refutation* [6]. To check whether a judgement $\phi \vdash P$ holds (i.e., whether the proposition P follows from the set of known assumptions ϕ), we assert all propositions in ϕ, assert $\neg P$, and check for unsatisfiability. If the solver returns UNSAT, then $\phi \wedge \neg P$ has no model, meaning P must hold under ϕ. If the solver returns SAT, a countermodel exists and the proof obligation fails—the type checker reports an error.

This approach means that the *discharge* of proof obligations is fully automatic: the programmer never constructs proof terms. However, the programmer must still supply the specifications from which these obligations arise—type annotations, function contracts, and loop invariants. The automation lies in checking, not in specification.

In Veritas, the SMT solver is invoked at several points: subtyping checks between refinement types (where the solver verifies that one predicate implies another), array bounds verification (where the solver must prove that an index lies within $[0, N)$), and function contract verification (where preconditions and postconditions are checked at call sites and return points respectively).

2.1.5 Type-preserving compilation and DTAL

Conventional compilers erase types after type checking, discarding the safety guarantees established by the type system. The generated machine code is then trusted on the basis that the compiler is correct, an as-

sumption that is difficult to verify for large, complex compilers. Type-preserving compilation [14] retains type information through every intermediate representation in the pipeline. This enables the property that the compiled output can be independently verified without trusting the compiler. If a bug in the compiler produces incorrect code, the type annotations on the output will be inconsistent, and the verifier will reject it.

The idea of shipping machine code together with a checkable proof of its safety was formalised by Necula as proof-carrying code (PCC) [5]. In PCC, the code producer (i.e., compiler) generates both executable code and a safety proof; the code consumer (e.g., an operating system loader) checks the proof against a safety policy before executing the code. The proof checker is small and can be verified more quickly and easily than the entire compiler. Type-preserving compilation can be seen as a particular instance of PCC where the 'proof' takes the form of type annotations.

Xi and Harper introduced DTAL as a target for type-preserving compilation from DML (Dependent ML) [11] and Xanadu [15], a dependently typed imperative programming language with C-like syntax. DTAL augments a conventional assembly language with two key forms of type annotation:

- **Label signatures**: each basic block entry point is annotated with existentially quantified index variables and the types expected for each register on entry, enabling modular, block-by-block verification.
- **Register types**: types may depend on index variables, so that a register's type tracks its exact value (e.g., `int`(i)) or the size of an array it points to (e.g., `int array`(n)).

The following excerpt from Xi and Harper [8] shows the loop header of a DTAL copy function. The label signature binds index variables m, n, i and declares register types in terms of them:

```
loop:  {m:nat, n:nat | m <= n, i:nat}
       [r1: int array(m), r2: int array(n),
        r3: int(m), r4: int(i)]
       sub   r5, r4, r3       // r5 <- i - m
       bgte  r5, finish       // if i >= m, exit
       load  r5, r1(r4)       // safe: i < m
       store r2(r4), r5       // safe: i < n (since m <= n)
       add   r4, r4, 1
       jmp   loop
```

The branch `bgte r5, finish` refines the type state: on the fall-through path, the type system knows $i < m$, which makes the subsequent `load` and `store` provably safe without any runtime bounds check. Xi and Harper proved that well-typed DTAL programs cannot violate the constraints of the type system.

Veritas implements its own DTAL variant based on Xi and Harper's design, extended to support the Veritas refinement type system and targeting x86-64. The compiler translates the source language through a typed intermediate representation (TIR) to DTAL, preserving refinement types at each stage, and finally emits x86-64 machine code as an ELF executable. Hereafter, 'DTAL' refers to this Veritas-specific variant unless otherwise noted; the original is referred to as 'Xi and Harper's DTAL'.

2.2 Related work and Design Influences

The design of Veritas makes use of several established approaches to software verification. Rather than relying on a single paradigm, it synthesises techniques from functional verification, systems programming, and proof-carrying code.

2.2.1 Refinement Types and Automated Verification

The use of an SMT solver to discharge type-level proof obligations is heavily inspired by Liquid Haskell [16] and the broader Liquid Types framework [6]. While Liquid Haskell targets a pure, lazy functional language,

Veritas applies this automated refinement checking to an imperative systems language. Similarly, Dafny [17] relies on SMT solvers (via the Boogie intermediate language) to verify imperative function contracts and loop invariants. However, both Dafny and Liquid Haskell erase their proofs during compilation. Veritas mirrors the automated reasoning of these tools but preserves the resulting constraints down to the assembly level.

2.2.2 Foundational and Dependent Verification

At the more expressive end of the spectrum are systems based on full dependent types or separation logic. F* [18] supports dependent types and can extract code to C or WebAssembly. RefinedC [19] applies refinement and ownership types to existing C code, producing foundational, machine-checked proofs in Coq. While these systems offer stronger theoretical guarantees (foundational proofs rather than SMT-based trust), they generally require the programmer to construct explicit proof terms or interact with a proof assistant. Veritas restricts its logic to the decidable fragments supported by Z3, so that proof obligations are discharged automatically once specifications are provided.

2.2.3 Verifiable Systems Languages

The most direct predecessor to Veritas's goals is ATS (Applied Type System) [20], which also targets systems programming using dependent types with zero runtime overhead. The difference lies in the compilation architecture. ATS performs all theorem proving at compile time and then compiles to standard C, meaning the C compiler and the ATS compiler are both in the trusted computing base. In contrast, Veritas uses the architecture of DTAL. By preserving types through to the backend, Veritas ensures that the target code can be audited independently.

Figure 2.1 summarises the trade-off between expressiveness and automation across the systems discussed above.

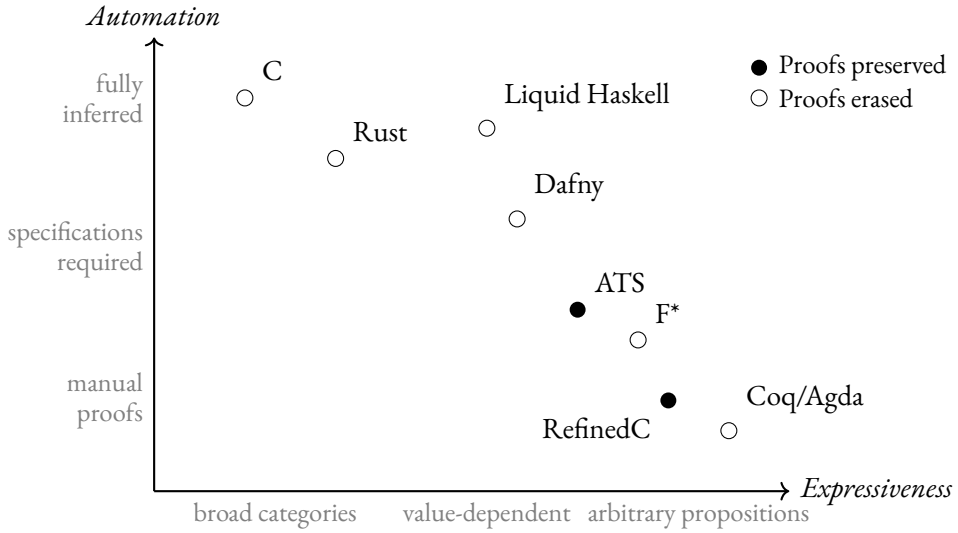

Figure 2.1: Verification systems positioned by expressiveness of their type/specification language and degree of automation. Filled markers indicate systems that preserve type-level proofs through compilation; hollow markers indicate systems that erase them.

2.3 Design decisions

This section describes the key architectural decisions made during the preparation phase. The full syntax and formal typing rules for both the source language and DTAL are presented in Chapter 3 alongside their implementation.

2.3.1 Compiler architecture

A central architectural decision is that the compiler is *untrusted*. Rather than proving the compiler correct (as in CompCert [4]), correctness is enforced by a standalone verifier that independently re-checks the compiler's output. This means bugs anywhere in the compiler cannot compromise safety—they can only cause the verifier to reject the output. Figure 2.2 illustrates this architecture: type annotations flow through every intermediate representation, and the trust boundary separates the untrusted compiler from the small trusted core.

2.3.2 Trust model and TCB reduction

The project was designed in phases that progressively reduced the trusted computing base (TCB). Each phase moved the verification boundary further down the pipeline, removing components from the set of code that must be correct for the safety guarantee to hold:

1. **Virtual DTAL verification**: the verifier checks DTAL with virtual registers. The TCB includes the verifier, Z3, register allocation, instruction selection, and the x86-64 encoder.
2. **Physical DTAL verification**: the verifier checks DTAL *after* register allocation. Register allocation and instruction selection are removed from the TCB. The TCB is now the verifier, Z3, and a trivial direct encoder.
3. **Runtime and bare-metal**: the runtime is kept small enough to audit by inspection, and a bare-metal target removes the OS from the TCB entirely. These are described in Sections 3.8 and 3.9.

2.3.3 Source language design

Veritas is an imperative language with C-like syntax, taking inspiration from Xi and Harper's Xanadu. It is designed to be minimal but expressive. The language supports integers, booleans, fixed-size arrays, mutable and immutable variables, for-loops, while-loops, and functions. The type system extends simple types with singleton types, refinement types, function contracts (with bounded quantifiers), and loop invariants.

2.3.4 DTAL target language design

The target language is a variant of Xi and Harper's Dependently Typed Assembly Language [8], extended to support the Veritas refinement type system and targeting x86-64. DTAL resembles a RISC assembly language augmented with type annotations on registers, constraint assertions embedded in the instruction stream, and declared type states at basic block entry points.

A key design decision was to define DTAL with its own *physical register model* (registers `r0`–`r12`, `sp`, `fp`, `lr`) that maps directly to x86-64 registers. This allows verification to occur *after* register allocation: the register allocator is untrusted, and its output is verified before the final (trivial) encoding to x86-64.

2.3.5 Design challenges

Combining refinement types with imperative features, SSA-based compilation, and an untrusted-compiler architecture raised several interacting design challenges that required extending or departing from the approaches in the literature.

Mutable variables and refinement types. Refinement type systems such as Liquid Haskell operate over immutable bindings, where a variable's type is fixed at its definition. Supporting mutation introduces a soundness hazard: after `x = e`, any refinement mentioning `x` may be invalidated. The solution adopted—*master types*—required designing the mutable context Δ to track both a current (refinable) type and a fixed master type, with a subtyping check on every assignment.

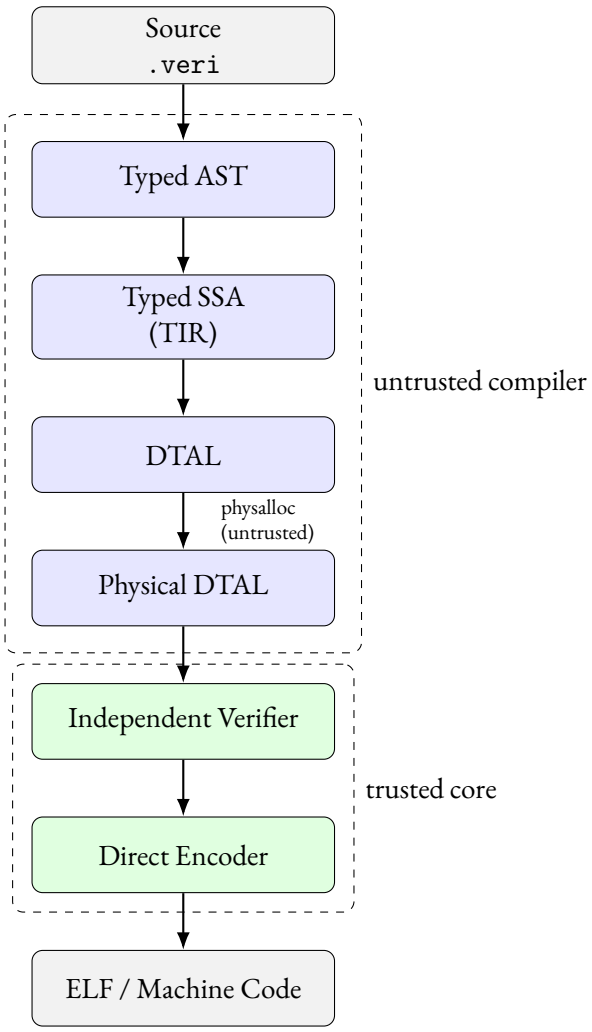

Figure 2.2: Veritas compilation pipeline and trust boundary.

Verification after register allocation. The initial design verified DTAL with virtual registers and then trusted the x86-64 lowering (∼1,100 lines of register allocation, instruction selection, and encoding). This placed a large, error-prone component inside the TCB. Redesigning the pipeline to verify *after* register allocation required defining a physical DTAL register model, splitting the monolithic lowering into an untrusted physical allocation pass and a trivial direct encoder, and extending the verifier to handle physical instructions (prologues, spills, division expansion). This reduced the trusted backend code to ∼300 lines.

Type preservation across SSA join points. At phi nodes in the SSA intermediate representation, two singleton types from different branches (e.g., $\texttt{int}(3)$ and $\texttt{int}(7)$) must be joined. The naive approach—widening to the base type `int`—loses all refinement information, making downstream bounds proofs impossible. The solution was to attach *existential constraints* to phi nodes, encoding that the value satisfies a constraint without fixing it to a concrete value. This mechanism was not part of Xi and Harper's original DTAL design and required extending both the TIR and the verifier.

SMT encoding of array properties. Verifying array-manipulating programs requires reasoning about array contents, which involves quantified formulae. These fall outside the decidable quantifier-free fragment of linear integer arithmetic. The design restricted quantifiers to *bounded* forms ($\textbf{forall}\ x\ \textbf{in}\ a..b$), which Z3's E-matching heuristics handle reliably in practice, at the cost of excluding unbounded quantification. This trade-off was accepted to maintain the automatability of verification.

2.4 Requirements analysis

A systematic requirements analysis was conducted during the preparation phase to define the scope of the project. The requirements were categorised using the MoSCoW method (Must-have, Should-have, Could-have, Won't-have) to prioritise features and ensure the core objectives of type-preserving compilation and independent verification were met.

2.4.1 Must-have

To successfully demonstrate the paradigm of proof-carrying code and independent assembly verification, the system must include:

- A high-level systems language featuring essential constructs (variables, control flow, functions, arrays).
- A dependent type system supporting refinement types and singleton types to express spatial memory safety invariants.
- A compiler frontend with a bidirectional type checker that delegates proof obligations (e.g., array bounds) to an SMT solver.
- A Dependently Typed Assembly Language (DTAL) target capable of embedding type states and constraint assertions.
- A compiler backend that preserves type information through intermediate representations and outputs verified DTAL.
- A standalone DTAL verifier capable of re-verifying the safety of the generated assembly without access to the original source code.

2.4.2 Should-have

To enhance the practical utility and robustness of the toolchain, the project should ideally include:

- End-to-end executable generation, translating the verified DTAL into native x86-64 machine code and packaging it as a freestanding ELF binary.
- Advanced function contracts, including postconditions (`ensures`) and quantified specifications (`forall`, `exists`).
- A comprehensive automated test suite capable of systematically testing both compilation success for safe programs and targeted rejection of unsafe programs.

2.4.3 Could-have

Features that would extend the system's capabilities but were not critical to the core proof-of-concept included:

- Backend compiler optimisations (such as copy propagation and dead code elimination) operating over the typed intermediate representation.
- Extended language semantics, such as structs, generics, and a full ownership model (similar to Rust).
- An interpreter for executing DTAL programs directly.

2.4.4 Won't-have

To maintain a manageable scope and clear focus on structural safety, several ambitious extensions were explicitly excluded:

- A foundational, machine-checked proof of compiler correctness (e.g., via Coq or Isabelle). The project's trust model explicitly treats the compiler as untrusted.
- Verification of concurrency properties, such as data race freedom or deadlock avoidance.
- Verification of information flow control or cryptographic protocols.

2.5 Evaluation plan

The evaluation aims to assess three dimensions: *correctness*, *overhead* (what is the cost of verification?), and *expressiveness* (what can the type system express, and how large is the trusted computing base?).

Correctness will be tested via suites of positive programs (expected to compile, verify, and execute correctly) and negative programs (expected to be rejected with specific errors). Verifier agreement—whether the independent DTAL verifier accepts every program the compiler produces—will serve as a cross-check of the trust architecture.

Overhead will be measured quantitatively using the following metrics:

- Compilation and verification time (with and without verification enabled)
- SMT query count and cumulative solver time (frontend and verifier)
- Output binary size compared to equivalent C programs compiled with GCC
- Runtime execution time compared to C equivalents

Compilation timings will be collected using built-in instrumentation (the `-bench` flag emits per-stage JSON). Runtime benchmarking will use `hyperfine` for statistical rigour. A benchmarking harness will automate data collection across the test suite.

Expressiveness will be assessed qualitatively: whether non-trivial algorithms can be verified, the annotation burden relative to executable code, and the proportion of the codebase within the trusted computing base.

The language and its performance will also be compared against the existing verification systems surveyed in Section 2.2. Language comparisons will examine equivalent programs across systems (where possible), considering syntax verbosity, annotation requirements, and the kinds of properties each system can express. Performance comparisons will draw on published overhead figures from Liquid Haskell, Dafny, and similar tools to contextualise Veritas's verification cost.

2.6 Software engineering methodology

2.6.1 Methodology

Development followed a hybrid approach. Initial planning resembled a waterfall model in which the compiler pipeline modules, source and target language features and specifications were defined. Software development then followed a spiral approach: the pipeline was built end-to-end with minimal features first (integer arithmetic only), then progressively extended with arrays, mutable variables, refinement types, function contracts, and quantifiers. Work items were planned for two-week sprints and I kept track of tasks using a Kanban board on Jira. Each feature was accompanied by test cases in both positive (should compile) and negative (should produce a specific error) categories. The test suite features unit tests for each significant module and integration tests between the components of the pipeline.

[TODO: final Gantt chart diagram here]

Version control was managed using Git, employing a trunk-based development strategy. Features and bug fixes were developed in short-lived branches before being integrated into a central master branch. To maintain code quality and ensure continuous correctness, I set up an automated Continuous Integration (CI) pipeline via GitHub Actions, which triggered on every push to the remote repository. This pipeline enforced formatting and linting standards (using the `cargo fmt` and `cargo clippy` tools), validated the suite of example programs, and executed the test suite across both Ubuntu Linux and macOS environments; this covers two primary platforms on which systems programming toolchains are used in practice and ensures the compiler and verifier implementations remain portable and correct across OS environments, libc implementations (relevant to the C-based implementation of Z3), and toolchain versions. Testing was done against both stable and nightly Rust toolchains; stable ensures the project builds with the version most users would have, while nightly provides early warning of upcoming compiler changes or deprecations, and is necessary if any features behind features gates are used.

I also attended thrice weekly standups with other Computer Science students, where we gave brief reports on our project status and upcoming work items.

2.6.2 Tools and Technologies

The project was implemented in Rust, chosen for its strong type system, pattern matching, and memory safety guarantees without garbage collection.

The Z3 SMT solver used for the frontend typechecker and DTAL verifier was accessed via the `z3` Rust crate, which provided high-quality, idiomatic Rust bindings, compared to potential alternatives like CVC5 that have less mature Rust ecosystem support. As mentioned in Section 2.1.4, Z3 is one of the most widely-deployed, well-audited SMT solvers. In Veritas, Z3 is used by the DTAL verifier and thus forms part of the trusted computing base; it is important to minimise the risk introduced by this trust—using an obscure solver would weaken this argument. Another prominent reason for using Z3 is that it natively supports the theories featured in Veritas: quantifier-free LIA for refinement subtyping, and bounded quantifiers for array specifications.

Another notable crate used to develop the Veritas compiler was the `im` crate, which provides persistent (immutable) data structures for the typing context, enabling $O(\log n)$ cloning at control-flow branches. This is important for the functional-style context threading used in bidirectional type checking e.g., where each branch of an **if-else** receives its own copy of the context.

2.7 Starting point

I had introductory knowledge of dependent type theory (DTT) and proof assistant and verification languages, and was introduced to refinement types via Xi and Harper's DTAL. No external code was used for the compiler or verifier. To successfully complete the project, I had to gain proficiency in several new tools, algorithms and theories: the Rust programming language, typechecking and proof assertion via SMT, Z3 theories, x86-64 machine code encoding, and bidirectional type checking. Key resources included the papers from Xi et al. on DML, Xanadu, and DTAL as well as other existing systems and their theories.

3 Implementation

This chapter describes the implementation of the Veritas toolchain: a type-preserving compiler, an independent DTAL verifier, a compact runtime, and a bare-metal unikernel target. The chapter begins with the formal definitions of the source language (Section 3.1) and DTAL target language, then presents the compilation pipeline (Sections 3.5–3.7), the verifier (Section 3.10), and the runtime and bare-metal components (Sections 3.8–3.9). Section 3.11 traces a binary search program through the full pipeline as a worked example, and Section 3.12 provides a repository overview.

3.1 The Veritas language

This section presents the syntax and formal typing rules of the Veritas source language. These define the contract that the compiler must uphold: any program accepted by the type checker satisfies the properties encoded in its types.

3.1.1 Syntax

The abstract syntax of Veritas is defined by the following grammar:

$$c ::= \underline{n} \mid \mathsf{true} \mid \mathsf{false} \qquad \textbf{(Base values)}$$

$$\mathsf{aop} ::= \mathsf{+} \mid \mathsf{-} \mid \mathsf{*} \mid \mathsf{/} \mid \mathsf{\%} \qquad \textbf{(Arithmetic operators)}$$

$$\mathsf{cop} ::= \mathsf{==} \mid \mathsf{!=} \mid \mathsf{<} \mid \mathsf{<=} \mid \mathsf{>} \mid \mathsf{>=} \qquad \textbf{(Comparison operators)}$$

$$\mathsf{lop} ::= \mathsf{\&\&} \mid \mathsf{||} \mid \mathsf{==>} \qquad \textbf{(Logical operators)}$$

$$\begin{aligned} e ::= {} & c \mid x \mid e\ \mathsf{aop}\ e \mid e\ \mathsf{cop}\ e \mid e\ \mathsf{lop}\ e \mid \mathsf{!}\ e \mid \mathsf{-}\ e \mid f(\overrightarrow{e}) \mid \\ & x[e_i] \mid \mathsf{if}\ e_{\mathsf{cond}}\ \{B\}\ \mathsf{else}\ \{B'\} \mid [e_{\mathsf{val}}; e_{\mathsf{size}}] \end{aligned} \qquad \textbf{(Expressions)}$$

$$\begin{aligned} P ::= {} & e\ \mathsf{cop}\ e \mid P\ \mathsf{lop}\ P \mid \mathsf{!}\ P \mid \\ & \mathsf{forall}\ x\ \mathsf{in}\ e_{\mathsf{start}}..e_{\mathsf{end}}\ \{P\} \mid \mathsf{exists}\ x\ \mathsf{in}\ e_{\mathsf{start}}..e_{\mathsf{end}}\ \{P\} \end{aligned} \qquad \textbf{(Propositions)}$$

$$\begin{aligned} S ::= {} & \mathsf{let}\ x : T = e; \mid \mathsf{let\ mut}\ x : T = e; \mid x = e; \mid x[e_i] = e; \mid \mathsf{return}\ e; \mid e; \mid \\ & \mathsf{if}\ e_{\mathsf{cond}}\ \{B\}\ (\mathsf{else}\ \{B'\})^? \mid \mathsf{for}\ x\ \mathsf{in}\ e_{\mathsf{start}}..e_{\mathsf{end}}\ (\mathsf{invariant}\ P)^?\ \{B\} \mid \\ & \mathsf{while}\ e_{\mathsf{cond}}\ (\mathsf{invariant}\ P)^?\ \{B\} \end{aligned} \qquad \textbf{(Statements)}$$

$$D ::= \mathsf{fn}\ f(x_1 : T_1, \ldots)\ (\to T_{\mathsf{ret}})^?\ (\mathsf{requires}\ P)^?\ (\mathsf{ensures}\ P)^?\ \{B\} \qquad \textbf{(Function definitions)}$$

$$\mathsf{Prog} ::= D^* \qquad \textbf{(Program)}$$

$$B ::= S^*\ e^? \qquad \textbf{(Blocks)}$$

$$T ::= \mathsf{unit} \mid \mathsf{int} \mid \mathsf{bool} \mid [T; N] \mid \&T \mid \&\mathsf{mut}\ T \mid \mathsf{int}(N) \mid \{x : \mathsf{int} \mid P\} \qquad \textbf{(Types)}$$

The metavariables used in the grammar are as follows: n ranges over integer literals; x, f range over identifiers (variables and function names); N ranges over integer expressions that must reduce to a statically known constant (used for array sizes and singleton types). P is the specification language used in refinement predicates, function contracts, and loop invariants, and additionally provides bounded quantifiers. A block B is a sequence of zero or more statements followed by an optional trailing expression; when the trailing expression is present, its value is the block's result. The superscript $(\cdot)^?$ denotes an optional clause; $(\cdot)^*$ denotes zero or more repetitions.

3.1.2 Typing contexts

The typing context is a four-tuple $(\phi, \Gamma, \Delta, \Sigma_F)$:

- ϕ: A set of refinement propositions known to be true (assumptions from branch conditions, preconditions, and refinement types).
- Γ: An immutable variable environment mapping names to types $(x : \tau)$.
- Δ: A mutable variable environment mapping names to pairs of a *current type* and a *master type* $(x : (\tau_c, \tau_m))$. We write $(x : \tau_c) \in \Delta$ to extract the current type and $(x : \tau_m) \in \Delta$ for the master type.
- Σ_F: A global function signature environment (read-only after an initial pass).

The master type τ_m is set at declaration and never changes; the current type τ_c may be refined through control flow but must remain a subtype of τ_m. The interaction between master types and mutable variables is described in Section 3.5.2.

3.1.3 Judgement forms

Throughout the typing rules, τ is used as the metavariable for types. The syntax grammar uses T to define the syntactic forms of types as they appear in source code; in the judgements below, these are all simply τ.

The type system uses the following judgements:

- **Synthesis:** $\phi; \Delta; \Gamma \vdash e \Rightarrow (\phi'; \Delta'; \tau)$ — synthesise type τ for expression e, potentially updating the context.
- **Checking:** $\phi; \Delta; \Gamma \vdash e \Leftarrow \tau : (\phi'; \Delta')$ — check that e has type τ.
- **Subtyping:** $\phi \vdash \tau_1 <: \tau_2$ — τ_1 is a subtype of τ_2 under assumptions ϕ.
- **Provability:** $\phi \vdash P$ — proposition P is valid under assumptions ϕ (decided by SMT).

The checking and synthesis judgements are connected by a subsumption rule. If an expression synthesises type τ_1 and the expected type is τ_2, checking succeeds when $\tau_1 <: \tau_2$:

$$\frac{\phi; \Delta; \Gamma \vdash e \Rightarrow (\phi'; \Delta'; \tau_e) \quad \phi' \vdash \tau_e <: \tau}{\phi; \Delta; \Gamma \vdash e \Leftarrow \tau : (\phi'; \Delta')} \quad \text{(T-Sub)}$$

3.1.4 Subtyping rules

The subtyping relation $\phi \vdash \tau_1 <: \tau_2$ is defined by structural rules (reflexivity, singleton-to-int widening, refined-to-int widening, array covariance, master type transparency — listed in full in Appendix A) and two rules that invoke the SMT solver. In both, the propositions in ϕ are asserted as assumptions, the goal is negated, and the solver is checked for unsatisfiability:

$$\frac{\phi \vdash P[N/x]}{\phi \vdash \mathsf{int}(N) <: \{x : \mathsf{int} \mid P\}} \quad \text{(Sub-Singleton-Refine)}$$

$$\frac{\phi \wedge P_1 \vdash P_2}{\phi \vdash \{x : \mathsf{int} \mid P_1\} <: \{x : \mathsf{int} \mid P_2\}} \quad \text{(Sub-Refine-Refine)}$$

3.1.5 Expression typing rules

The expression typing rules use two auxiliary functions on types. $\mathsf{val}(\tau)$ extracts the value expression from a type: $\mathsf{val}(\mathsf{int}(N)) = N$ for singletons, and $\mathsf{val}(\{x : \mathsf{int} \mid P\}) = x$ for refined types. $\mathsf{prop}(\tau)$ extracts a proposition: $\mathsf{prop}(\mathsf{int}(N)) = (v = N)$, $\mathsf{prop}(\{x : \mathsf{int} \mid P\}) = P$, and $\mathsf{prop}(\tau) = \top$ for unrefined types.

Standard rules for literals, variable lookup, comparisons, logical operators, and array creation are given in Appendix A. Some of the more non-trivial rules are described below.

Arithmetic operations use a join-op function that performs constant folding when both operands are singletons (e.g., $\mathsf{join\text{-}op}(+, \mathsf{int}(1), \mathsf{int}(2)) = \mathsf{int}(3)$) and otherwise synthesises a refinement type via SMT refinement synthesis (e.g., $\{v : \mathsf{int} \mid v = e_L + e_R\}$):

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e_L \Rightarrow (\phi_1; \Delta_1; \tau_L) \quad \phi_1 \vdash \tau_L <: \mathsf{int} \\ \phi_1; \Delta_1; \Gamma \vdash e_R \Rightarrow (\phi_2; \Delta_2; \tau_R) \quad \phi_2 \vdash \tau_R <: \mathsf{int} \\ \tau_{\mathsf{res}} = \mathsf{join\text{-}op}(\mathsf{aop}, \tau_L, \tau_R)\end{array}}{\phi_0; \Delta_0; \Gamma \vdash e_L \ \mathsf{aop}\ e_R \Rightarrow (\phi_2; \Delta_2; \tau_{\mathsf{res}})} \quad \text{(T-Op)}$$

Array indexing requires a bounds proof obligation, verified by the SMT solver:

$$\frac{\begin{array}{c}(x : [\tau_{\mathsf{elem}}; N]) \in (\Gamma \cup \Delta_0) \\ \phi_0; \Delta_0; \Gamma \vdash e_i \Rightarrow (\phi_1; \Delta_1; \tau_i) \quad \phi_1 \vdash \tau_i <: \mathsf{int} \\ \phi_1 \wedge \mathsf{prop}(\tau_i) \vdash 0 \le \mathsf{val}(\tau_i) < N \quad \text{(Bounds proof)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash x[e_i] \Rightarrow (\phi_1; \Delta_1; \tau_{\mathsf{elem}})} \quad \text{(T-Array-Idx)}$$

Conditional expressions check both branches under the condition and its negation, then join the resulting contexts using the join operation defined in Section 3.1.7:

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e_{\mathsf{cond}} \Rightarrow (\phi_1; \Delta_1; \tau_c) \quad \phi_1 \vdash \tau_c <: \mathsf{bool} \\ \phi_1 \wedge \mathsf{prop}(\tau_c); \Delta_1; \Gamma \vdash B \Rightarrow (\phi_2; \Delta_2; \tau_2) \\ \phi_1 \wedge \neg\mathsf{prop}(\tau_c); \Delta_1; \Gamma \vdash B' \Rightarrow (\phi_3; \Delta_3; \tau_3) \\ (\phi_{\mathsf{join}}, \Delta_{\mathsf{join}}, \tau_{\mathsf{join}}) = \mathsf{join}(\phi_2, \Delta_2, \tau_2, \phi_3, \Delta_3, \tau_3)\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{if}\ e_{\mathsf{cond}}\ B\ \mathsf{else}\ B') \Rightarrow (\phi_{\mathsf{join}}; \Delta_{\mathsf{join}}; \tau_{\mathsf{join}})} \quad \text{(T-If-Expr)}$$

Function calls verify argument types and the precondition (a proof obligation discharged by the SMT solver):

$$\frac{\begin{array}{c}\Sigma_F(f) = ((x_i : \tau_i)_{i=1}^{k} \to \tau_{\mathsf{ret}}\ \mathsf{requires}\ P_{\mathsf{req}}) \\ \phi_0; \Delta_0; \Gamma \vdash \vec{e} \Rightarrow (\phi_k; \Delta_k; \vec{\sigma}) \\ \forall i \in 1..k,\ \phi_k \vdash \sigma_i <: \tau_i \\ \phi_k \wedge \mathsf{prop}(\vec{\sigma}) \vdash P_{\mathsf{req}}[\mathsf{val}(\vec{\sigma})/\vec{x}] \quad \text{(Precondition proof)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash f(\vec{e}) \Rightarrow (\phi_k; \Delta_k; \tau_{\mathsf{ret}})} \quad \text{(T-Call)}$$

3.1.6 Statement typing rules

Standard rules for blocks, let bindings, array assignment, if-statements, and while loops are given in Appendix A. Some of the more non-trivial rules are described below.

Assignment checks that the new value conforms to the master type:

$$\frac{\begin{array}{c}(x : \tau_{\mathsf{master}}) \in \Delta_0 \\ \phi_0; \Delta_0; \Gamma \vdash e \Rightarrow (\phi_1; \Delta_1; \tau_e) \\ \phi_1 \vdash \tau_e <: \tau_{\mathsf{master}}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (x = e;) : (\phi_1; \Delta_1[x \mapsto \tau_e]; \Gamma)} \quad \text{(T-Assign)}$$

For loops bind the loop variable to the immutable context with bound propositions added to ϕ, and require the loop invariant (if present) to hold on entry and be preserved by the body. The invariant is checked at entry by substituting e_{start} for i, and preservation is checked after the body by substituting $i + 1$ for i:

$$\frac{\begin{array}{c}\phi_0 \wedge P_I[e_{\mathsf{start}}/i] \quad \text{(Invariant holds on entry)} \\ \phi_0 \wedge P_I \wedge e_{\mathsf{start}} \le i < e_{\mathsf{end}};\ \Delta_0;\ (\Gamma, i : \mathsf{int}) \vdash B : (\phi_1; \Delta_1; \Gamma) \\ \phi_1 \vdash P_I[(i+1)/i] \quad \text{(Invariant preserved by body)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{for}\ i\ \mathsf{in}\ e_{\mathsf{start}}..e_{\mathsf{end}}\ \{B\}) : (\phi_0 \wedge P_I; \Delta_0; \Gamma)} \quad \text{(T-For)}$$

where P_I is the invariant proposition (or $\top$ if no invariant is specified).

3.1.7 Context joining

At control-flow merge points (after **if-else** and loops), the typing contexts from each branch must be joined. The join operation computes a least upper bound for each mutable variable's type and intersects the proposition sets:

- Same singletons: $\mathsf{int}(N) \sqcup \mathsf{int}(N) = \mathsf{int}(N)$.
- Different singletons: $\mathsf{int}(N_1) \sqcup \mathsf{int}(N_2) = \mathsf{int}$ where $N_1 \neq N_2$ (widen to base type).
- Refined types: $\{v \mid P\} \sqcup \{v \mid Q\} = \{v \mid P \vee Q\}$ (disjoin predicates).
- Singleton and refined: promote the singleton to $\{v \mid v = N\}$, then disjoin with the refined type's predicate.
- Refined and unrefined: $\{v \mid P\} \sqcup \mathsf{int} = \mathsf{int}$ (widen to base type).
- Arrays: $[\tau_1; N] \sqcup [\tau_2; N] = [\tau_1 \sqcup \tau_2; N]$ (join element types covariantly).
- Propositions (ϕ): intersect (keep only propositions provable in both branches).

When join is used in expression rules (e.g., T-If-Expr), the same type-joining rules apply to the result types of both branches.

3.1.8 Function contracts

Programmers may declare preconditions (**requires** P) and postconditions (**ensures** P) on functions, where P is a proposition that may reference parameters and (for postconditions) the special variable `result`.

At each call site, the type checker verifies $\phi \vdash P_{\mathsf{req}}[\overline{e_i/x_i}]$: the precondition with actual argument expressions substituted for formal parameters (see rule T-Call). At each return statement, the return expression is synthesised, checked against the declared return type, and the postcondition is verified:

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e \Rightarrow (\phi_1; \Delta_1; \tau_e) \quad \phi_1 \vdash \tau_e <: \tau_{\mathsf{ret}} \\ \phi_1 \vdash P_{\mathsf{ens}}[e/\texttt{result}] \quad \text{(Postcondition proof)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{return}\ e;) : (\phi_1; \Delta_1; \Gamma)} \quad \text{(T-Return)}$$

3.1.9 Quantified specifications

The constraint language supports bounded quantifiers and implication in specification positions:

- **forall** x **in** $e_{\text{start}}..e_{\text{end}}$ $\{P\}$: asserts $\forall x.\ e_{\text{start}} \leq x < e_{\text{end}} \Rightarrow P$.
- **exists** x **in** $e_{\text{start}}..e_{\text{end}}$ $\{P\}$: asserts $\exists x.\ e_{\text{start}} \leq x < e_{\text{end}} \wedge P$.
- $P \Rightarrow Q$ (implication): enables conditional specifications.

Quantifiers may only appear in specification positions (preconditions, postconditions, refinement predicates), not in runtime code. The type checker enforces this restriction.

An important feature is *refinement lifting*: an array typed as $[\{v : \mathsf{int} \mid v \geq 0\}; N]$ automatically satisfies the quantified constraint $\forall i \in [0, N).\ arr[i] \geq 0$. This is achieved by extracting a quantified proposition from the array's element type and adding it to the context when the array is in scope.

3.2 The DTAL target language

As introduced in Section 2.1.5, Veritas targets its own DTAL variant based on Xi and Harper's design [8]. In their original work, Xi and Harper formally established the soundness of DTAL, proving that well-typed programs cannot violate memory safety or the constraints of the type system. Veritas builds on this theoretical foundation: the compiler emits DTAL annotated with type information, and a standalone verifier re-derives the types from scratch. If the compiler introduces an error, the annotations will be inconsistent and the verifier will reject the program. This section describes the concrete instruction set and typing judgements of the Veritas DTAL.

3.2.1 Instruction set

The DTAL instruction set resembles a standard RISC assembly language but is augmented with type annotations and constraint assertions. It operates over a set of virtual registers (which are later allocated to x86-64 physical registers) and linear memory. Instructions fall into six categories:

- **Data movement:** `mov` r_d, r_s, `mov` r_d, n, `load` r_d, `[`r_b, r_i`]`, `store [`r_b, r_i`]`, r_s
- **Arithmetic:** `binop` r_d, r_{s1}, r_{s2} (add, sub, mul, div, mod), `addimm` r_d, r_s, n, `not` r_d, r_s
- **Comparison:** `cmp` r_1, r_2, `cmp` r_1, n, `setcc` r_d, *`cond`* (materialise boolean from flags)
- **Control flow:** `jmp` L, `branch` *`cond`* L, `call` L, `ret`
- **Stack:** `push` r_s, `pop` r_d, `alloca` r_d, n (stack-allocate an array of n elements)
- **Annotations:** `type` r`:` τ (type annotation), `assert` C (constraint assertion)

The annotation instructions are the distinguishing feature of DTAL. Every instruction carries a type annotation on its destination register, and `assert` instructions embed constraint assertions (e.g., bounds proofs, loop invariants) directly into the instruction stream. These annotations serve as verification evidence: the compiler emits them, and the verifier checks them independently.

After register allocation, additional physical instructions are emitted (`prologue`, `epilogue`, spill loads/stores, `cqo`/`idiv` for division) which are verified but not relevant to the formal typing rules. Crucially, the independent verifier operates on this post-regalloc *physical* DTAL rather than the virtual-register SSA form, meaning it also validates the correctness of register allocation and spilling. This strengthens the safety guarantee: the register allocator, like the rest of the compiler, is untrusted.

The final step translates verified physical DTAL to x86-64 machine code and packages it as a freestanding ELF binary. Rather than emitting assembly text and invoking an external assembler (e.g., NASM or GAS), Veritas encodes x86-64 instructions directly as bytes. This is a deliberate design choice to minimise the trusted computing base: an external assembler would be a large, unverified component whose correctness must be assumed. The Veritas encoder comprises three small modules: a one-to-one mapping from physical DTAL instructions to x86-64 instructions, a byte encoder, and an ELF generator. The mapping

is deliberately trivial—each DTAL instruction translates to exactly one x86-64 instruction (or a small fixed sequence for prologues/epilogues)—making the entire path amenable to manual audit.

The choice of x86-64 as the target architecture is a pragmatic one: the development machine runs x86-64 Linux, enabling direct execution and testing of compiled binaries without emulation. Targeting a different ISA (e.g., AArch64) would require replacing only the instruction mapping, byte encoder, and register definitions. The DTAL instruction set, type system, verifier, and everything upstream would remain unchanged.

3.2.2 DTAL type system

The DTAL type system mirrors the source-level type system but operates over registers rather than variables. Types include:

- **Base types:** int, bool, unit
- **Singleton types:** $\mathsf{int}(e)$ where e is an index expression (e.g., $\mathsf{int}(5)$, $\mathsf{int}(r_1 + r_2)$)
- **Refined types:** $\{x : \mathsf{int} \mid C\}$ where C is a constraint over x
- **Existential types:** $\exists x.\ \mathsf{int}(x)\ \textbf{where}\ C$ — the result of arithmetic on refined (non-singleton) operands
- **Array types:** $[\tau; e]$ where τ is the element type and e is the size

Index expressions form the language of type indices: constants, register names, arithmetic ($+, -, \times, \div, \%$), and $\mathsf{select}(arr, i)$ for array element access. Constraints are first-order formulae over index expressions supporting comparison, logical connectives, and bounded quantifiers.

3.2.3 DTAL typing judgements

DTAL type checking operates over a *type state* (Γ, Φ) where Γ maps registers to their current types and Φ is a set of constraints known to hold at a given program point. The core judgement for instruction typing is $\Phi; \Gamma \vdash \mathsf{instr} \Rightarrow \Gamma'$, meaning that executing instr under constraints Φ and register state Γ produces a new register state Γ'.

Move Immediate Loading a constant into a register assigns it a singleton type:

$$\frac{}{\Phi; \Gamma \vdash \mathsf{mov}\ r_d, n \Rightarrow \Gamma[r_d \mapsto \mathsf{int}(n)]} \quad \text{(D-MovImm)}$$

Arithmetic When both operands have singleton types, the result is a singleton capturing the exact value. In the general case (refined or existential operands), the result is an existential type:

$$\frac{\Gamma(r_1) = \mathsf{int}(e_1) \quad \Gamma(r_2) = \mathsf{int}(e_2)}{\Phi; \Gamma \vdash \mathsf{add}\ r_d, r_1, r_2 \Rightarrow \Gamma[r_d \mapsto \mathsf{int}(e_1 + e_2)]} \quad \text{(D-BinOp-Singleton)}$$

$$\frac{\Gamma(r_1) = \{a : \mathsf{int} \mid \varphi_1\} \quad \Gamma(r_2) = \{b : \mathsf{int} \mid \varphi_2\}}{\Phi; \Gamma \vdash \mathsf{add}\ r_d, r_1, r_2 \Rightarrow \Gamma[r_d \mapsto \exists c.\ \mathsf{int}(c)\ \textbf{where}\ \exists a, b.\ \varphi_1 \wedge \varphi_2 \wedge c = a + b]} \quad \text{(D-BinOp-Refined)}$$

Memory Access (Bounds Checking) A memory load is only well-typed if Φ can prove that the index is within bounds. The result type is the array's element type:

$$\frac{\Gamma(r_b) = [\tau; N] \quad \Phi \vdash 0 \leq e_i < N}{\Phi; \Gamma \vdash \mathsf{load}\ r_d, [r_b, r_i] \Rightarrow \Gamma[r_d \mapsto \tau]} \quad \text{(D-Load)}$$

where e_i is the index expression extracted from the type of r_i. Stores are handled symmetrically, with an additional mechanism: the verifier maintains *array versions* so that each store creates a fresh logical name for the array. This enables the SMT solver to reason about array contents using store/select axioms without conflating values written at different program points.

Conditional Branching A comparison instruction (`cmp` r_1, r_2) records the operands; a subsequent conditional branch (`branch` *cond* L) refines the constraint context on both paths. If the operands have index expressions e_1 and e_2:

- Taken branch (to L): $\Phi' = \Phi \wedge (e_1 \ \mathsf{cond} \ e_2)$
- Fall-through: $\Phi' = \Phi \wedge \neg(e_1 \ \mathsf{cond} \ e_2)$

This is used to verify bounds checks: a comparison followed by a branch-if-greater-or-equal exits the loop, and the fall-through path continues with the assumption that the index is within bounds.

These rules are representative; the verifier handles all instruction variants listed in Section 3.2.1. The complete set of DTAL typing rules is given in Appendix B.

3.3 Pipeline overview

Figure 3.1 summarises the compilation pipeline. Each named representation is shown in sequence, and arrows indicate the transformations between them. Type annotations are preserved through TAST, TIR, and DTAL, and are no longer needed after DTAL verification and final encoding to x86-64.

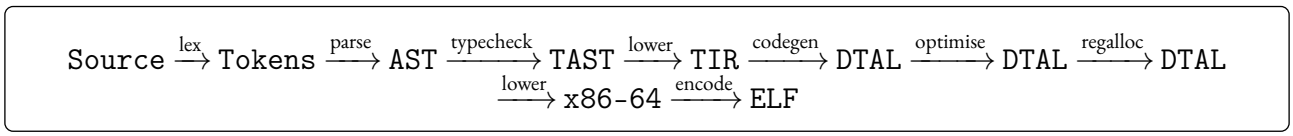

Figure 3.1: The Veritas compilation pipeline. Representations carrying type annotations are shown in bold.

3.4 Usage

Veritas programs are written in `.veri` source files. The compiler is invoked from the command line:

```
veritas <source.veri> [OPTIONS]
```

By default, the compiler type-checks the source, lowers it through TIR and DTAL, and prints the DTAL output to stdout. The most common options are:

- `-o <file>`: compile to a native ELF executable.
- `-verify`: verify the DTAL output before code generation.
- `-verify-only`: verify DTAL and exit without producing an executable.
- `-O`: enable all optimisations (copy propagation and dead code elimination).
- `-v`: verbose mode, showing each compilation stage.
- `-tir`: print the TIR (SSA) intermediate representation.

For example, compiling and verifying a program end-to-end:

```
veritas bubble_sort.veri --verify -O -o bubble_sort
```

The resulting executable is a statically linked, freestanding ELF binary. The return value of `main` becomes the process exit code. The runtime provides five intrinsics overall: `print_int(n)`, `print_char(c)`, `read_int()`, `port_in(port)`, and `port_out(port, val)`. On the bare-metal target, the type checker admits only the port-I/O intrinsics.

A separate mode, `-verify-dtal`, accepts a standalone `.dtal` file and verifies it without requiring the source, demonstrating that the verifier operates independently of the compiler frontend. The `-target -bare-metal` flag produces a Multiboot-compliant ELF for execution directly on QEMU.

3.5 Frontend

3.5.1 Lexing and parsing

The lexer and parser are built using the `chumsky` parser combinator library. The lexer converts source text into a stream of tokens annotated with source spans; the parser transforms this stream into an untyped AST.

The frontend makes use of *two expression parsers*: a full parser (`expr_parser`) that includes if-expressions and function calls, and a restricted variant (`expr_parser_for_types`) used inside refinement type annotations. The restricted parser excludes constructs that would be ambiguous or meaningless in a type position, while supporting the arithmetic and comparison operators needed for refinement predicates. This separation avoids ambiguity in the grammar without requiring a separate expression language for types.

3.5.2 Type checker

The type checker is the most substantial frontend component. It implements the bidirectional type system described in Chapter 2, using Z3 as an SMT oracle.

Architecture

The type checker is structured around the `TypingContext` $(\phi, \Gamma, \Delta, \Sigma_F)$ defined in Section 3.1. Each judgement receives a context and returns an updated one, following a functional style. This design is made practical by the `im` crate, which provides persistent (structurally shared) data structures: cloning a context at a control-flow branch is $O(\log n)$ rather than $O(n)$.

The type checker operates in two passes. The first collects all function signatures into Σ_F, enabling mutual recursion and allowing functions to be defined in any order. The second type-checks each function body against its signature.

Bidirectional checking

The two modes of the bidirectional system are implemented as separate functions. `synth_expr` implements synthesis: given an expression, it returns the most precise type the checker can infer. Integer literals produce singleton types (`int`(n)), variable lookups return the type from Γ or Δ, and arithmetic operations use the `join_op` function described below. `check_expr` implements checking: given an expression and an expected type, it synthesises a type and verifies the subtyping relationship via the SMT oracle.

Statement checking (`check_stmts`) updates the typing context according to each statement's effect: `let` bindings extend Γ or Δ, assignments update the current type in Δ, and loops apply invariant-based reasoning as described in Chapter 2.

Singleton propagation and refinement synthesis

When both operands of an arithmetic operation are singletons, the type checker performs constant folding at the type level: $\texttt{int}(a) \oplus \texttt{int}(b) = \texttt{int}(a \oplus b)$. This avoids an SMT query entirely.

When one or both operands are not singletons, the checker performs *refinement synthesis*: it constructs a fresh refinement type $\{v : \texttt{int} \mid v = e_L \oplus e_R\}$ that captures the exact arithmetic relationship between the result and its operands. The bound variable v is given a fresh name (e.g., `_synth_0`) to avoid capture.

This synthesised refinement enables downstream code to reason about the result precisely—for instance, to verify that an array index computed as `lo + (hi - lo) / 2` lies within bounds, given that `lo` ≥ 0 and `hi` $< n$.

Mutable variables and master types

Mutable variables present a challenge for refinement types: an assignment `x = e` changes the value of `x`, potentially invalidating refinements that mention it. Veritas addresses this with *master types*. When a mutable variable is declared as `let mut x: T = e`, the type T is recorded as the variable's master type T_m. The variable's *current type* T_c may be narrowed through assignments and control flow (e.g., after `if x > 0`, the current type narrows to $\{v : \texttt{int} \mid v > 0\}$), but every assignment must satisfy $\tau_e <: T_m$. At control-flow joins, current types from each branch are merged using the join operation, but the result must remain a subtype of the master type.

Context joining

At control-flow merge points, the typing contexts from each branch are reconciled using the join operation defined in Section 3.1.7. The implementation uses `im`'s persistent maps to efficiently compute the join without mutating either branch's context.

SMT oracle

The SMT oracle wraps Z3 and provides a single entry point: `is_provable`(ϕ, P), which returns whether proposition P is valid under assumptions ϕ. Internally, it creates a fresh Z3 context, translates each proposition in ϕ to a Z3 expression, asserts them, asserts $\neg P$, and checks for unsatisfiability.

Translation requires mapping between the Veritas type system and Z3's sort system. Integer expressions map to Z3 integer sort; boolean expressions to Z3 boolean sort. Arrays are represented using Z3's built-in array sort (`Array(Int, Int)`), where `select` corresponds to array indexing. Bounded quantifiers such as **forall** x **in** $e_1..e_2$ $\{P\}$ become $\forall x.\ (e_1 \leq x \wedge x < e_2) \Rightarrow P$ using Z3's native quantifier support. While this moves outside the decidable quantifier-free fragment, Z3's E-matching heuristics handle the bounded quantifiers that arise in practice.

An important feature is *refinement lifting*: when an array has a refined element type (e.g., $[\{v : \texttt{int} \mid v \geq 0\}; n]$), the oracle extracts a universally quantified proposition ($\forall i \in [0, n).\ arr[i] \geq 0$) and adds it to the assumption set. This allows element-level refinements to be used in quantified reasoning without the programmer restating them.

Array bounds and function contracts

Array bounds checking and function contract verification follow the formal rules T-Array-Idx, T-Call, and T-Return defined in Section 3.1. The implementation translates each rule directly: array accesses invoke the SMT oracle with the bounds obligation constructed from the index's synthesised type; call sites substitute actual arguments into the precondition; and return statements substitute the return expression for the special variable `result` in the postcondition.

3.5.3 Error reporting

Type errors are represented as structured values (`TypeError` variants) carrying source spans, expected and actual types, and contextual information. A separate reporting module formats these into diagnostics using the `ariadne` library, highlighting the relevant source location and explaining the failed proof obligation, including the propositions that were assumed and the goal that could not be proved.

3.6 Middle end

3.6.1 Typed Intermediate Representation (TIR)

The TIR is a typed SSA (Static Single Assignment) intermediate representation. Each function is a control-flow graph of basic blocks; each block contains a sequence of three-address instructions followed by a terminator (jump, conditional branch, or return). Every instruction carries type annotations from the source-level type checker, and every virtual register is defined exactly once.

The choice of SSA was motivated by three properties: liveness analysis (required for register allocation) is straightforward, the one-definition property makes type annotations unambiguous, and phi nodes provide natural points for refinement type joining. At control-flow merge points, phi nodes select between values from different predecessor blocks.

A key extension beyond standard SSA is *existential constraints on phi nodes*. Each phi node may carry an optional existential constraint of the form $\exists v.\ \texttt{int}(v)\ \textbf{where}\ C(v)$, which encodes that the phi's value satisfies constraint C without fixing a concrete value. For loop induction variables, this captures the loop bounds: $\exists v.\ \texttt{int}(v)\ \textbf{where}\ (v \geq \mathit{start} \wedge v < \mathit{end})$. For mutable variables with refined types, it preserves the refinement predicate across iterations. This mechanism avoids the information loss that would occur if phi nodes were simply widened to the base type.

3.6.2 Lowering from TAST to TIR

The lowering pass translates the typed AST to TIR. A `LoweringContext` maintains a mapping from source-level variable names to virtual registers and tracks the current basic block under construction.

Control flow

Conditionals are lowered in the standard SSA style: the condition branches to 'then' and 'else' blocks, which rejoin at a merge block. The only Veritas-specific detail is that each branch edge carries a logical constraint derived from the condition, so an `if x > 0` yields assumptions $x > 0$ and $\neg(x > 0)$ on the respective successors. Phi nodes are inserted only for variables whose assigned register differs between the two branches.

For-loops use the usual entry, header, body, and exit structure. The induction variable is represented by a header phi node with incoming values from the entry edge and the back-edge, and this phi carries an existential constraint recording the loop range $[\mathit{start}, \mathit{end})$. Mutable variables in scope also receive header phi nodes so that loop-carried values and loop-invariant values are explicit to the verifier.

While-loops are lowered similarly, except that the condition is re-evaluated in the header. Because the condition need not be a simple interval test, the lowerer cannot attach the same explicit range constraint used for `for` induction variables; verification instead relies on the translated branch condition and any programmer-supplied invariant.

Constraint propagation

Branch conditions are translated into the TIR's constraint language, which supports linear arithmetic comparisons, logical connectives, quantifiers, and array indexing. This translation converts source-level expressions into `Constraint` values via a recursive traversal: comparisons map directly, logical operators produce constraint connectives, and quantifiers map to bounded quantified constraints.

A critical detail is *variable-to-register substitution*: constraints in the TAST refer to source-level variable names, but the TIR and DTAL operate on virtual register names. During lowering, the context maintains a substitution map from variable names to register names (e.g., `"lo"` $\mapsto$ `"v11"`), and all emitted constraints are rewritten accordingly. This ensures constraints remain valid through register allocation and DTAL verification.

Loop invariants are emitted as `AssertConstraint` instructions at the start of the loop body, after applying the variable-to-register substitution. These assertions are the TIR counterpart of the invariant veri-

fication in the type checker: the type checker proves the invariant holds; the TIR records it as an assertion that the verifier will independently re-check.

3.7 Backend

3.7.1 DTAL code generation

The code generator translates TIR to DTAL, the dependently typed assembly language described in Section 2.1.5. At this stage DTAL still uses virtual registers, but each basic block is annotated with a *type state*: a mapping from registers to types together with the constraints known to hold at that program point. These annotations are the core of the type-preserving design, because they give the standalone verifier enough information to re-check the program without access to the source.

Most TIR instructions lower directly to DTAL. Phi nodes are implemented as parallel copies on predecessor edges, and array operations become address computations plus loads or stores with the relevant bounds constraints attached.

3.7.2 Optimisation

Two optimisation passes operate on the DTAL representation before register allocation:

Copy propagation identifies instructions of the form `mov r1, r2` and replaces subsequent uses of `r1` with `r2`, eliminating redundant copies introduced by phi-node lowering.

Dead code elimination removes instructions whose destination register is never used.

Both passes iterate to a fixed point, as copy propagation may expose new dead code and vice versa. The passes must preserve type annotations: when a copy is propagated, the type of the source register must be compatible with all uses of the destination.

[TODO:]

3.7.3 Register allocation

Register allocation is implemented using standard liveness analysis followed by graph colouring. The default allocator is a Chaitin–Briggs style allocator over the 11 allocatable general-purpose x86-64 registers (excluding `rsp`, `rbp`, `rax`, `rdx`, and `r11`). Registers live across function calls are constrained to callee-saved registers (`rbx`, `r12`–`r15`), reflecting the calling convention. For comparison and experimentation, the implementation also includes a linear-scan allocator selectable via the `VERITAS_LS=1` environment variable.

3.7.4 Physical allocation

The register allocator produces a mapping from virtual to physical registers; the *physical allocation pass* applies this mapping to produce a DTAL program that uses only physical registers. This is the central transformation in the default pipeline, and it is *untrusted*—its output is verified by the DTAL verifier before code generation proceeds.

DTAL defines 16 physical registers: `r0`–`r12` (general purpose), `sp` (stack pointer), `fp` (frame pointer), and `lr` (link register / return value). These map directly to x86-64 registers:

Beyond simple register renaming, the physical allocation pass performs several x86-specific transformations:

- **Function prologues and epilogues**: emits explicit `Prologue` and `Epilogue` instructions that push/pop callee-saved registers and allocate a 16-byte-aligned stack frame.
- **Division expansion**: a single virtual `div` instruction is expanded to the x86-64 sequence `mov rax, lhs; cqo; idiv rhs; mov dst, rax` (or `rdx` for modulo), since `idiv` requires its operands in the `rdx:rax` pair.

DTAL	x86-64	Role	Saved by
r0–r5	rdi, rsi, rdx, rcx, r8, r9	Arguments 1–6	Caller
r6–r7	r10, r11	Scratch	Caller
r8–r12	rbx, r12–r15	General	Callee
lr	rax	Return value	Caller
sp	rsp	Stack pointer	—
fp	rbp	Frame pointer	—

Table 3.1: DTAL physical register mapping to x86-64.

- **Caller-saved register management**: at each call site, caller-saved registers that are live across the call are spilled to the stack before argument setup and restored after the call returns.
- **Parameter move cycle resolution**: when function parameters must be shuffled between registers (e.g., parameter 0 in `rdi` must move to `rsi`, while parameter 1 in `rsi` must move to `rcx`), the pass detects cycles and breaks them using `r11` as a scratch register.
- **Constraint remapping**: all type constraints are rewritten from virtual register names (`v3`) to physical register names (`r0`), so the verifier can check the physical program.

3.7.5 Direct encoding

The *direct encoder* translates physically-allocated DTAL to x86-64 machine code. Because the physical allocation pass has already performed all x86-specific expansions (division, calling convention, prologues), the encoder is a near-trivial 1:1 mapping: each DTAL instruction maps to exactly one x86-64 instruction (or a short fixed sequence for prologues and epilogues). The entire module is 309 lines in the current implementation.

This simplicity is deliberate and architecturally significant. The initial implementation used a monolithic x86-64 lowering pass that performed register allocation, instruction selection, and encoding in a single trusted step. By splitting this into an untrusted physical allocation pass (verified by the DTAL verifier) and a trivial encoder, the trusted code for the final translation was reduced from ∼1,100 lines to ∼300 lines. The encoder is small enough to audit by inspection, which is important because it is the only backend component that remains in the trusted computing base.

3.7.6 Binary encoding and ELF generation

The direct encoder emits x86-64 machine code from physical DTAL, using a two-pass scheme to resolve label offsets. The resulting byte stream is then wrapped in a minimal ELF executable with entry point `main`; for the Linux target this is an ELF64 binary, while the bare-metal target uses a separate Multiboot-compatible layout described below.

3.8 Runtime

The compiler provides a small set of runtime intrinsics that are linked into every executable. These are implemented as hand-assembled x86-64 machine code totalling approximately 210 bytes, prepended to the user's compiled code in the ELF output.

Five intrinsics are provided:

- `print_int(n: int)`: converts a signed 64-bit integer to its decimal string representation and writes it to stdout followed by a newline, using the Linux `write` syscall.
- `print_char(c: int)`: writes a single byte to stdout.
- `read_int() -> int`: reads a decimal integer from stdin using the Linux `read` syscall, handling an optional leading minus sign.

- `port_in(port: int) -> int`: reads a byte from an x86 I/O port using the `in` instruction.
- `port_out(port: int, val: int)`: writes a byte to an x86 I/O port using the `out` instruction.

The first three intrinsics use Linux syscalls and are available only when compiling for the Linux target. The port I/O intrinsics use hardware instructions directly and are available on both Linux and bare-metal targets. The type checker enforces this distinction: a bare-metal program that calls `print_int` is rejected at compile time rather than failing at runtime.

All intrinsics follow the System V AMD64 calling convention (first argument in `rdi`, return value in `rax`), so they are callable from compiled Veritas code without any marshalling. Because the runtime is hand-assembled and part of the trusted computing base, its correctness is verified by inspection rather than by the DTAL verifier.

3.9 Bare-metal target and unikernel

The `-target-bare-metal` flag produces a Multiboot-compliant ELF32 that can boot directly on x86-64 hardware or QEMU without an operating system. This extends the trust model by removing the OS from the trusted computing base: the only software running on the machine is the verified Veritas program and the small runtime.

3.9.1 Bootstrap

The bare-metal binary begins with a bootstrap blob of approximately 200 bytes that performs the transition from the Multiboot entry point (32-bit protected mode) to 64-bit long mode:

1. A Multiboot header allows GRUB or QEMU's `-kernel` flag to load the binary.
2. The bootstrap sets up a minimal page table (identity-mapping the first 8 MB using four 2 MB huge pages), enables PAE and long mode via the appropriate control register writes, and performs a far jump into 64-bit code.
3. A GDT (Global Descriptor Table) with 64-bit code and data segments is loaded.
4. The stack pointer is set to a fixed address (`0x200000`) and `main` is called.
5. After `main` returns, the bootstrap enters a `hlt` loop.

The bootstrap is hand-assembled and part of the trusted computing base, but at ∼200 bytes it is small enough to audit line by line.

3.9.2 Verified serial driver

To demonstrate end-to-end verified I/O on bare metal, the project includes a serial driver written entirely in Veritas. The driver programs the UART 16550 controller (the standard PC serial port at I/O base `0x3F8`) using the `port_in` and `port_out` intrinsics:

- `serial_init()` configures the baud rate (38400), line format (8N1), and FIFO.
- `serial_write(c: int)` spins on the Line Status Register until the transmit buffer is empty, then writes a byte to the data register.
- `serial_print_int(n: int)` recursively prints a signed integer to the serial port.

The entire driver is type-checked and compiled through the verified DTAL pipeline. When compiled with `-target-bare-metal -verify -o serial`, the resulting binary boots in QEMU and outputs to the serial console:

```
qemu-system-x86_64 -kernel serial -serial stdio -display none
```

This demonstrates the project's long-term direction: a verified unikernel where the application, driver, and boot code are all compiled from a single verified source, with the OS removed from the trusted computing base entirely. Figure 3.2 illustrates the architecture of a bare-metal Veritas binary.

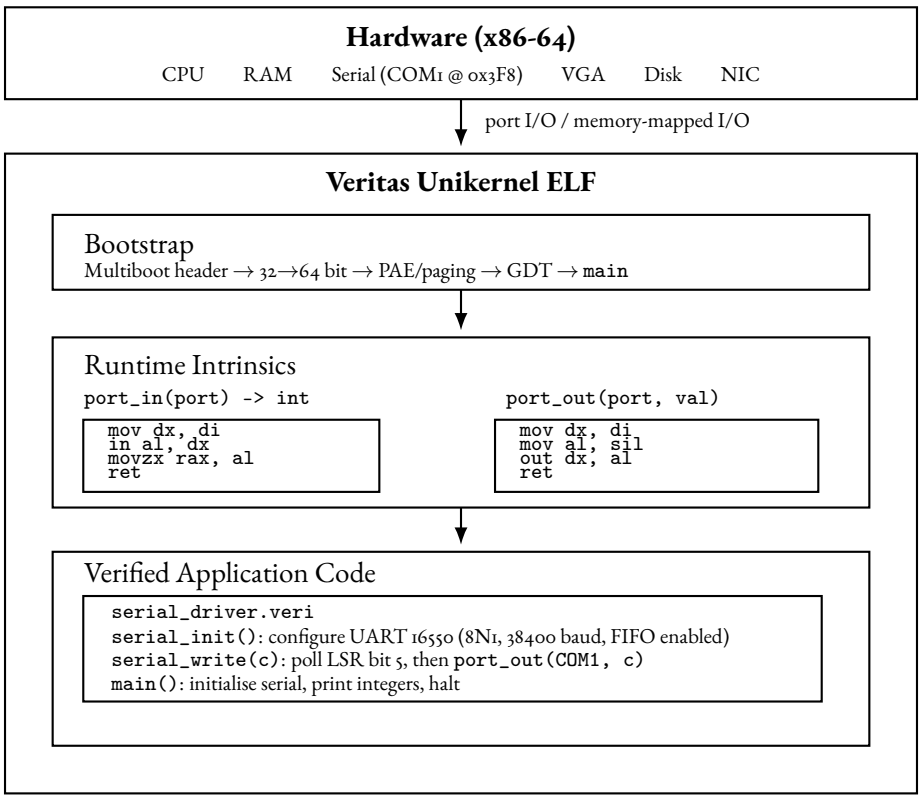

Figure 3.2: Architecture of a bare-metal Veritas unikernel. The binary contains a Multiboot bootstrap, a small set of hand-assembled runtime intrinsics, and verified application code compiled through the DTAL pipeline.

3.10 Independent verifier

A key contribution of the type-preserving architecture is that the DTAL output can be verified independently of the compiler. The verifier (approximately 3,100 lines) re-checks the DTAL program from scratch, using only the type annotations embedded in the assembly. It follows the derivation-based verification approach of Xi and Harper [8]: types are *re-derived* from instruction semantics, not trusted from compiler annotations.

3.10.1 Verification algorithm

The verifier processes each function independently. For each basic block, it:

1. Initialises the type state from the block's declared entry state (register types and constraints).
2. Symbolically executes each instruction, *re-deriving* the result type from the operand types according to the instruction's semantics.
3. At each constraint assertion (`.assert`), invokes its own SMT oracle to verify that the constraint follows from the current type state.
4. At block exits (jumps, branches, returns), performs *state coercion*: verifies that the computed exit state is compatible with the declared entry state of each successor block.

The re-derivation in step 2 is central to the untrusted-compiler model. For example, `mov rd, 5` always derives `rd : int(5)` from the immediate value, regardless of any annotation the compiler may have emitted. Similarly, `add rd, r1, r2` derives `rd : int(r1 + r2)` using symbolic index expression algebra. The compiler's annotations serve only as a cross-check; if the re-derived type conflicts with the annotation, the verifier reports an error.

3.10.2 State coercion at control-flow edges

State coercion is how the verifier checks compatibility across block boundaries. When block A jumps to block B, it must show that A's exit state is a subtype of B's declared entry state. This reduces to two checks:

- **Register type subtyping**: for each register in B's entry state, A's exit type must be a subtype. The subtyping relation is constraint-aware: $\texttt{int}(n) <: \texttt{int}$ always holds, and $\texttt{int}(n) <: \exists v.\ \texttt{int}(v)$ **where** $C(v)$ holds if $C(n)$ is provable from A's constraints.
- **Constraint entailment**: every constraint declared in B's entry state must be provable from A's exit constraints.

For conditional branches, the verifier first refines the exit state on each edge. For example, `cmp r1, r2` followed by `jl target` contributes $\texttt{r1} < \texttt{r2}$ on the taken edge and $\texttt{r1} \geq \texttt{r2}$ on the fall-through edge before coercion is checked against the respective successor states.

3.10.3 Array bounds verification

Array loads and stores are the most critical instructions for the verifier. For each `load rd, [base + offset]`:

1. The verifier checks that `base` has an array type $[\tau; N]$.
2. It constructs the bounds obligation $0 \leq \texttt{offset} < N$.
3. It attempts to prove this obligation, first via fast syntactic checks (e.g., is the constraint already in the context?), then via the SMT oracle.
4. If the proof succeeds, it derives the element type τ for `rd` and emits a *select axiom*: $\texttt{rd} = arr[\texttt{offset}]$, linking the loaded value to the array's logical representation.

Stores additionally emit a *write axiom* ($arr_{new}[\texttt{offset}] = \texttt{value}$) and a *frame axiom* ($\forall k \neq \texttt{offset}.\ arr_{new}[k] = arr_{old}[k]$). The array is tracked with a version number that increments on each store, ensuring that the SMT solver can distinguish array states before and after mutations.

3.10.4 Interprocedural contract verification

When the verifier encounters a `call` instruction, it performs three checks:

1. **Precondition**: the callee's precondition, with formal parameters substituted for actual argument registers, must be provable from the caller's current constraints.
2. **Return type**: the callee's declared return type is assigned to the destination register.
3. **Postcondition**: the callee's postcondition (similarly substituted) is added to the caller's constraint set for use by subsequent instructions.

This enables modular verification: each function is checked in isolation, relying on contracts rather than inlining.

3.10.5 SMT oracle

The verifier uses its own Z3 instance, completely independent of the one used during compilation. The translation follows the same proof-by-refutation approach as the compiler's oracle (Section 3.5.2): assert the context, assert the negated goal, and check for unsatisfiability.

To improve performance, constraint provability uses a two-tier approach. A fast syntactic check handles common cases (constraint already in context, simple entailment patterns) in $O(n)$ where n is the context size. The SMT solver is invoked only when the syntactic check is inconclusive. This is important because the verifier issues many more queries than the compiler—one per instruction rather than one per subtyping check.

3.10.6 Verification after register allocation

The compiler supports two pipeline modes. The *legacy pipeline* (`-legacy-pipeline`) verifies the virtual-register DTAL and then trusts the x86-64 lowering (register allocation, instruction selection, encoding). The *physical pipeline* (the default) defers verification until *after* register allocation has mapped virtual registers to physical x86-64 registers. In the physical pipeline, the verifier checks the physically-allocated DTAL, meaning that register allocation, spill insertion, and calling-convention lowering are all *untrusted*: any bug in these passes manifests as a type derivation mismatch or unprovable constraint in the verified output.

This is a deliberate architectural choice. Register allocation is one of the most error-prone compiler passes (it involves liveness analysis, graph colouring, spill heuristics, and calling-convention details), and verifying its output rather than trusting it substantially reduces the trusted computing base.

3.10.7 Trust model

The trust model and TCB reduction phases are described in Section 2.3. In summary, the verification architecture places the trust boundary between the physical DTAL verifier and the direct encoder: the entire compiler (frontend, middle end, backend, register allocator) is untrusted, and any bug in these components manifests as a verification failure rather than a safety violation.

3.11 Worked example: binary search

The preceding sections describe each pipeline stage in isolation. This section demonstrates the full pipeline by tracing a `binary_search` program, from source to verified DTAL. Binary search features most of the verification and constraint features of Veritas: quantified preconditions, refinement types, loop invariants, array bounds proofs, existential types, branch constraint refinement, and symbolic arithmetic.

3.11.1 Source language

The source program searches a sorted 10-element array for a target value, returning its index or -1 if not found:

```
fn binary_search(arr: [int; 10], target: int) -> int
    requires forall i in 0..9 { arr[i] <= arr[i + 1] }
{
    let mut lo: {v: int | v >= 0} = 0;
    let mut hi: int = 9;
    let mut result: int = -1;

    for iter in 0..20
        invariant lo >= 0 && hi < 10
    {
        if lo <= hi {
            let mid: int = lo + (hi - lo) / 2;
            let val: int = arr[mid];
            if val == target {
                result = mid;
                lo = hi + 1;
            } else {
                if val < target {
                    lo = mid + 1;
                } else {
                    hi = mid - 1;
```

```
                }
            }
        } else {
            let done: int = 0;
        }
    }

    result
}
```

Several features are worth highlighting:

- `requires forall i in 0..9 { arr[i] <= arr[i+1] }` — a quantified precondition asserting that the array is sorted in non-decreasing order. This is checked at every call site.
- `{v: int | v >= 0}` — a refinement type on `lo`, constraining it to be non-negative. This is the master type: every assignment to `lo` must satisfy it.
- `invariant lo >= 0 && hi < 10` — a loop invariant that bounds both endpoints. The type checker verifies this holds initially and is preserved by the loop body.
- `arr[mid]` — the critical array access. The type checker must prove $0 \leq \texttt{mid} < 10$ from the invariant and branch conditions.
- `for iter in 0..20` — the loop uses a bounded iteration counter. Although Veritas also supports `while` loops, the bounded `for` loop is useful here because it yields an explicit induction-variable range in the generated SSA and DTAL.

3.11.2 Frontend type checking

The type checker discharges three key proof obligations for this function. All are reduced to SMT queries of the form: assert the current context ϕ, assert $\neg G$ (the negated goal), and check for unsatisfiability.

Loop invariant: base case

Before the loop begins, the context contains $\texttt{lo} = 0$ (from the singleton type $\texttt{int}(0)$) and $\texttt{hi} = 9$ (from $\texttt{int}(9)$). The invariant to verify is:

$$\texttt{lo} \geq 0 \ \wedge \ \texttt{hi} < 10$$

Substituting the initial values gives $0 \geq 0 \wedge 9 < 10$, which is trivially satisfiable. The SMT oracle confirms this by showing that $\neg(0 \geq 0 \wedge 9 < 10)$ is unsatisfiable.

Loop invariant: inductive step

At the end of the loop body, the type checker must prove that the invariant is preserved. The context at this point includes the assumed invariant ($\texttt{lo} \geq 0 \wedge \texttt{hi} < 10$) plus the branch conditions taken during the iteration. In each branch:

- `val == target`: `lo` is set to `hi + 1`. Since $\texttt{hi} < 10$, we have $\texttt{lo} = \texttt{hi} + 1 \geq 1 \geq 0$, and `hi` is unchanged.
- `val < target`: `lo` is set to `mid + 1`. Since $\texttt{mid} \geq 0$ (from $\texttt{lo} \geq 0$ and the midpoint formula), $\texttt{lo} \geq 1 \geq 0$, and `hi` is unchanged.
- Otherwise: `hi` is set to `mid - 1`. Since $\texttt{mid} < 10$ (from $\texttt{hi} < 10$ and the midpoint formula), $\texttt{hi} < 10$, and `lo` is unchanged.

At the join point, the context-joining operation computes the least upper bound across all branches, and the SMT oracle verifies the invariant holds under the joined context.

Array bounds for arr[mid]

The array access `arr[mid]` occurs inside the branch guarded by `lo <= hi`. At this point the context contains:

$$\phi = \{\, \texttt{lo} \geq 0,\ \ \texttt{hi} < 10,\ \ \texttt{lo} \leq \texttt{hi},\ \ \texttt{mid} = \texttt{lo} + (\texttt{hi} - \texttt{lo})/2 \,\}$$

The proof obligation is $0 \leq \texttt{mid} < 10$. From the context:

- $\texttt{mid} \geq \texttt{lo} \geq 0$ (since $(\texttt{hi} - \texttt{lo})/2 \geq 0$ when $\texttt{lo} \leq \texttt{hi}$)
- $\texttt{mid} \leq \texttt{hi} < 10$ (since $\texttt{mid} = \texttt{lo} + (\texttt{hi} - \texttt{lo})/2 \leq \texttt{hi}$)

The SMT oracle asserts $\phi \wedge \neg(0 \leq \texttt{mid} \wedge \texttt{mid} < 10)$ and obtains UNSAT, confirming the bounds proof.

3.11.3 Lowering to TIR (SSA)

The lowering pass translates the function into a control-flow graph of 13 basic blocks. Figure 3.3 shows the structure, with blocks grouped by their role in the algorithm.

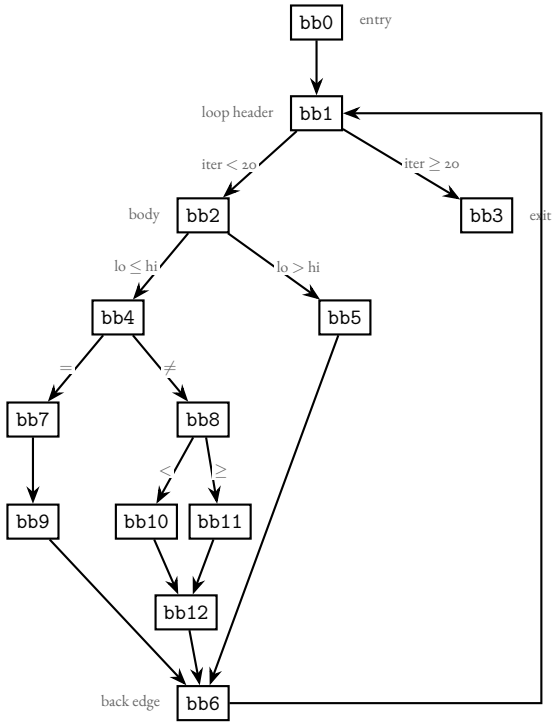

Figure 3.3: Control-flow graph of `binary_search`. Block bb4 contains the critical array access.

Key aspects of the TIR representation:

- The loop counter `iter` is assigned to virtual register `v8` with an existential type annotation:

$$\exists\, \texttt{v8}.\ \texttt{int}(\texttt{v8})\ \textbf{where}\ (\texttt{v8} \geq 0\ \wedge\ \texttt{v8} < 20)$$

  This captures the loop bounds without fixing a concrete value. The existential is opened at each use, making the bound constraints available for verification.
- Mutable variables (`lo`, `hi`, `result`) are represented by phi nodes at the loop header. Their types are widened to `int` at join points, since the concrete value depends on the iteration.
- An `.assert` directive at the entry of bb2 encodes the loop invariant $\texttt{v11} \geq 0 \wedge \texttt{v10} < 10$, where `v11` and `v10` are the registers holding `lo` and `hi` respectively.

3.11.4 DTAL code generation and verification

The DTAL output preserves the block structure of the TIR, augmented with entry state declarations, constraint assumptions, and type annotations on every instruction. Rather than showing all 13 blocks, this section traces the critical path: bb1 (loop header) → bb2 (body entry) → bb4 (the array access) → bb6 (back edge to loop header).

bb1: loop header

```
.binary_search_bb1:
    .type v8: exists v8. int(v8) where (v8 >= 0 && v8 < 20)
    .type v9: [int; 10]
    .type v10: int           ; hi
    .type v11: int           ; lo
    .type v12: int           ; result
    .type v13: int           ; target
    cmp v8, v7
    blt .binary_search_bb2
    jmp .binary_search_bb3
```

The verifier processes this block as follows. It opens the existential on v8, introducing a fresh witness satisfying $\texttt{v8} \geq 0 \wedge \texttt{v8} < 20$ into the constraint set. The comparison `cmp v8, v7` (where v7 holds 20) determines whether the loop continues. The conditional branch `blt` adds $\texttt{v8} < 20$ to bb2's context.

bb2: body entry — invariant verification

```
.binary_search_bb2:
    .assume v8 < v7
    .assert (v11 >= 0 && v10 < 10)
    cmp v11, v10
    ble .binary_search_bb4
    jmp .binary_search_bb5
```

The `.assert` directive is the loop invariant, now expressed over registers. The verifier must prove $\texttt{v11} \geq 0 \wedge \texttt{v10} < 10$ from the current constraint set. This constraint set includes all assumptions propagated from predecessor edges — specifically, the constraints verified at the bb0→bb1 and bb6→bb1 transitions. Entry state constraints are not trusted from the compiler; the verifier checks them independently at every incoming edge. The branch `ble` (branch if less-or-equal) adds $\texttt{v11} \leq \texttt{v10}$ (i.e., $\texttt{lo} \leq \texttt{hi}$) to bb4's context.

bb4: the array access

This is the most interesting block, containing the midpoint computation and the array load:

```
.binary_search_bb4:
    .assume v8 < v7
    .assume (v11 >= 0 && v10 < 10)
    .assume v11 <= v10
    sub v16, v10, v11        : int((v10 - v11))
    mov v17, 2               : int(2)
    div v18, v16, v17        : int(((v10 - v11) / 2))
    add v19, v11, v18        : int((v11 + ((v10 - v11) / 2)))
    load v20, [v9 + v19]     : int
```

The verifier re-derives the type of each instruction from its operands, following the Xi and Harper typing rules for DTAL [8]:

1. `sub v16, v10, v11` — from `v10 : int` and `v11 : int`, the verifier assigns `v16 : int`$((\texttt{v10} - \texttt{v11}))$.
2. `div v18, v16, v17` — yields `v18 : int`$(((\texttt{v10} - \texttt{v11})/2))$.
3. `add v19, v11, v18` — yields `v19 : int`$((\texttt{v11} + ((\texttt{v10} - \texttt{v11})/2)))$.
4. `load v20, [v9 + v19]` — the verifier must prove $0 \leq \texttt{v19} < 10$. The constraint set at this point is:

$$\{\ \texttt{v11} \geq 0,\ \ \texttt{v10} < 10,\ \ \texttt{v11} \leq \texttt{v10},\ \ \texttt{v19} = \texttt{v11} + (\texttt{v10} - \texttt{v11})/2\ \}$$

   The verifier sends $\phi \wedge \neg(0 \leq \texttt{v19} \wedge \texttt{v19} < 10)$ to Z3 and obtains UNSAT.

Note that the type annotations (e.g., `: int((v10 - v11))`) are emitted by the compiler but independently re-derived by the verifier. The verifier does not trust these annotations; it uses them only as a cross-check.

BB6: BACK EDGE — STATE COERCION

```
.binary_search_bb6:
    .type v47: int
    .type v48: int              ; hi (updated)
    .type v49: int              ; lo (updated)
    .type v50: int              ; result (updated)
    add v52, v8, v51            : int((v8 + 1))
    mov v8, v52                 : int
    mov v10, v48                : int
    mov v11, v49                : int
    mov v12, v50                : int
    jmp .binary_search_bb1
```

At the back edge, the updated values of `lo`, `hi`, and `result` are moved into the registers that bb1 expects. The loop counter is incremented: $\texttt{v52} = \texttt{v8} + 1$. The verifier checks that the exit state of bb6 is compatible with the declared entry state of bb1 — specifically, that `v52` satisfies the existential bound $\texttt{v8} \geq 0 \wedge \texttt{v8} < 20$ (after accounting for the increment), and that the loop invariant can be re-established from the updated register values.

3.12 Repository overview

The source code is organised as a single Rust crate with the following top-level module structure:

Original work. The entire compiler, verifier, and test suite were written from scratch for this project. No existing compiler or tutorial was used as a starting point. The project's design draws on the DTAL paper by Xi and Harper [8] for the typed assembly language and on the Liquid Types paper by Rondon et al. [6] for the refinement type checking approach, but the implementations are original.

External dependencies. Four third-party Rust crates are used:

- `z3` (v0.19, MIT): Rust bindings for the Z3 SMT solver [7] (MIT-licensed), used for proof obligations in both the type checker and verifier. Z3 is a substantial external system; its correctness is assumed.
- `chumsky` (v1.0.0-alpha.8, MIT): a parser combinator library used for the lexer and parser. It provides the parsing infrastructure, but all grammar rules are original.
- `im` (v15.1, MPL-2.0+): persistent (structurally shared) data structures, used for efficient context cloning during type checking.
- `ariadne` (v0.5, MIT): a diagnostic rendering library used for formatting error messages with source-location highlights. It handles the presentation layer only; all error detection and classification is original.

Directory	Lines	Responsibility
`src/common/`	624	Shared AST, typed AST, and type definitions
`src/frontend/`	6,498	Lexer, parser, and bidirectional type checker
`src/backend/tir/`	689	Typed intermediate representation (SSA)
`src/backend/lower/`	1,548	TAST to TIR lowering
`src/backend/dtal/`	2,387	DTAL syntax, constraints, and parser
`src/backend/codegen/`	1,004	TIR to DTAL code generation
`src/backend/optimise/`	935	Copy propagation and dead code elimination
`src/backend/regalloc/`	1,300	Liveness analysis and allocators
`src/backend/physalloc.rs`	1,171	Physical allocation and spill insertion
`src/backend/direct_encode.rs`	309	Direct physical-DTAL to x86-64 encoding
`src/backend/x86_64/`	2,699	Legacy trusted x86-64 lowering backend
`src/backend/elf/`	601	ELF generation, including bare-metal output
`src/backend/runtime.rs`	99	Hand-written runtime intrinsics
`src/backend/emit/`	411	DTAL text emitter
`src/verifier/`	4,825	Independent DTAL verifier
`src/main.rs`, `src/pipeline.rs`	1,046	CLI and pipeline orchestration
Total (excl. tests)	**26,017**	

Table 3.2: Repository structure and approximate line counts.

4 Evaluation

A Complete source-level typing rules

This appendix lists the complete set of typing rules for the Veritas source language. Rules marked with $\star$ are also presented in Chapter 3; they are repeated here for completeness.

Subsumption

$$\frac{\phi; \Delta; \Gamma \vdash e \Rightarrow (\phi'; \Delta'; \tau_e) \quad \phi' \vdash \tau_e <: \tau}{\phi; \Delta; \Gamma \vdash e \Leftarrow \tau : (\phi'; \Delta')} \quad \text{(T-Sub)}^\star$$

Subtyping

$$\frac{}{\phi \vdash \tau <: \tau} \quad \text{(Sub-Refl)}$$

$$\frac{}{\phi \vdash \mathsf{int}(N) <: \mathsf{int}} \quad \text{(Sub-Singleton-Int)}$$

$$\frac{}{\phi \vdash \{x : \mathsf{int} \mid P\} <: \mathsf{int}} \quad \text{(Sub-Refine-Int)}$$

$$\frac{\phi \vdash P[N/x]}{\phi \vdash \mathsf{int}(N) <: \{x : \mathsf{int} \mid P\}} \quad \text{(Sub-Singleton-Refine)}^\star$$

$$\frac{\phi \wedge P_1 \vdash P_2}{\phi \vdash \{x : \mathsf{int} \mid P_1\} <: \{x : \mathsf{int} \mid P_2\}} \quad \text{(Sub-Refine-Refine)}^\star$$

$$\frac{\phi \vdash \tau_1 <: \tau_2}{\phi \vdash [\tau_1; N] <: [\tau_2; N]} \quad \text{(Sub-Array)}$$

$$\frac{\phi \vdash \tau <: \sigma}{\phi \vdash \mathsf{master}(\tau) <: \sigma} \quad \text{(Sub-Master)}$$

Literals and variables

$$\frac{}{\phi; \Delta; \Gamma \vdash n \Rightarrow (\phi; \Delta; \mathsf{int}(n))} \quad \text{(T-Int)}$$

$$\frac{}{\phi; \Delta; \Gamma \vdash \mathsf{true} \Rightarrow (\phi; \Delta; \mathsf{bool})} \quad \text{(T-True)} \qquad \frac{}{\phi; \Delta; \Gamma \vdash \mathsf{false} \Rightarrow (\phi; \Delta; \mathsf{bool})} \quad \text{(T-False)}$$

$$\frac{(x : \tau) \in \Gamma}{\phi; \Delta; \Gamma \vdash x \Rightarrow (\phi; \Delta; \tau)} \quad \text{(T-Var-Imm)} \qquad \frac{(x : \tau_c) \in \Delta}{\phi; \Delta; \Gamma \vdash x \Rightarrow (\phi; \Delta; \tau_c)} \quad \text{(T-Var-Mut)}$$

Arithmetic

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e_L \Rightarrow (\phi_1; \Delta_1; \tau_L) \quad \phi_1 \vdash \tau_L <: \mathsf{int} \\ \phi_1; \Delta_1; \Gamma \vdash e_R \Rightarrow (\phi_2; \Delta_2; \tau_R) \quad \phi_2 \vdash \tau_R <: \mathsf{int} \\ \tau_{\mathsf{res}} = \mathsf{join\text{-}op}(\mathsf{aop}, \tau_L, \tau_R)\end{array}}{\phi_0; \Delta_0; \Gamma \vdash e_L \ \mathsf{aop}\ e_R \Rightarrow (\phi_2; \Delta_2; \tau_{\mathsf{res}})} \quad \text{(T-Op)}^\star$$

Comparison and logical operators

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e_L \Rightarrow (\phi_1; \Delta_1; \tau_L) \quad \phi_1 \vdash \tau_L <: \mathsf{int} \\ \phi_1; \Delta_1; \Gamma \vdash e_R \Rightarrow (\phi_2; \Delta_2; \tau_R) \quad \phi_2 \vdash \tau_R <: \mathsf{int}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash e_L \ \mathsf{cop}\ e_R \Rightarrow (\phi_2; \Delta_2; \mathsf{bool})} \quad \text{(T-Cmp)}$$

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e_L \Rightarrow (\phi_1; \Delta_1; \tau_L) \quad \phi_1 \vdash \tau_L <: \mathsf{bool} \\ \phi_1; \Delta_1; \Gamma \vdash e_R \Rightarrow (\phi_2; \Delta_2; \tau_R) \quad \phi_2 \vdash \tau_R <: \mathsf{bool}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash e_L \ \mathsf{lop}\ e_R \Rightarrow (\phi_2; \Delta_2; \mathsf{bool})} \quad \text{(T-Lop)}$$

Array creation

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e_{\mathsf{val}} \Rightarrow (\phi_1; \Delta_1; \tau_{\mathsf{val}}) \\ \phi_1; \Delta_1; \Gamma \vdash e_{\mathsf{size}} \Rightarrow (\phi_2; \Delta_2; \tau_{\mathsf{size}}) \\ \phi_2 \vdash \tau_{\mathsf{size}} <: \mathsf{int} \quad N = \mathsf{val}(\tau_{\mathsf{size}})\end{array}}{\phi_0; \Delta_0; \Gamma \vdash [e_{\mathsf{val}}; e_{\mathsf{size}}] \Rightarrow (\phi_2; \Delta_2; [\tau_{\mathsf{val}}; N])} \quad \text{(T-Array-Create)}$$

Array indexing

$$\frac{\begin{array}{c}(x : [\tau_{\mathsf{elem}}; N]) \in (\Gamma \cup \Delta_0) \\ \phi_0; \Delta_0; \Gamma \vdash e_i \Rightarrow (\phi_1; \Delta_1; \tau_i) \quad \phi_1 \vdash \tau_i <: \mathsf{int} \\ \phi_1 \wedge \mathsf{prop}(\tau_i) \vdash 0 \leq \mathsf{val}(\tau_i) < N \quad \text{(Bounds proof)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash x[e_i] \Rightarrow (\phi_1; \Delta_1; \tau_{\mathsf{elem}})} \quad \text{(T-Array-Idx)}^\star$$

Conditional expressions

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e_{\mathsf{cond}} \Rightarrow (\phi_1; \Delta_1; \tau_c) \quad \phi_1 \vdash \tau_c <: \mathsf{bool} \\ \phi_1 \wedge \mathsf{prop}(\tau_c); \Delta_1; \Gamma \vdash B \Rightarrow (\phi_2; \Delta_2; \tau_2) \\ \phi_1 \wedge \neg\mathsf{prop}(\tau_c); \Delta_1; \Gamma \vdash B' \Rightarrow (\phi_3; \Delta_3; \tau_3) \\ (\phi_{\mathsf{join}}, \Delta_{\mathsf{join}}, \tau_{\mathsf{join}}) = \mathsf{join}(\phi_2, \Delta_2, \tau_2, \phi_3, \Delta_3, \tau_3)\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{if}\ e_{\mathsf{cond}}\ B\ \mathsf{else}\ B') \Rightarrow (\phi_{\mathsf{join}}; \Delta_{\mathsf{join}}; \tau_{\mathsf{join}})} \quad \text{(T-If-Expr)}^\star$$

Function calls

$$\frac{\begin{array}{c}\Sigma_F(f) = ((x_i : \tau_i)_{i=1}^k \to \tau_{\mathsf{ret}}\ \mathsf{requires}\ P_{\mathsf{req}}) \\ \phi_0; \Delta_0; \Gamma \vdash \vec{e} \Rightarrow (\phi_k; \Delta_k; \vec{\sigma}) \\ \forall i \in 1..k,\ \phi_k \vdash \sigma_i <: \tau_i \\ \phi_k \wedge \mathsf{prop}(\vec{\sigma}) \vdash P_{\mathsf{req}}[\mathsf{val}(\vec{\sigma})/\vec{x}] \quad \text{(Precondition proof)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash f(\vec{e}) \Rightarrow (\phi_k; \Delta_k; \tau_{\mathsf{ret}})} \quad \text{(T-Call)}^\star$$

Blocks and bindings

$$\frac{}{\phi; \Delta; \Gamma \vdash \cdot : (\phi; \Delta; \Gamma)} \quad \text{(T-Block-Empty)}$$

$$\frac{\phi_0; \Delta_0; \Gamma_0 \vdash S : (\phi_1; \Delta_1; \Gamma_1) \quad \phi_1; \Delta_1; \Gamma_1 \vdash B : (\phi_2; \Delta_2; \Gamma_2)}{\phi_0; \Delta_0; \Gamma_0 \vdash (S\ B) : (\phi_2; \Delta_2; \Gamma_2)} \quad \text{(T-Block-Seq)}$$

$$\frac{\phi_0; \Delta_0; \Gamma \vdash e \Leftarrow \tau : (\phi_1; \Delta_1)}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{let}\ x : \tau = e;) : (\phi_1; \Delta_1; \Gamma, x : \tau)} \quad \text{(T-Let)}$$

$$\frac{\phi_0; \Delta_0; \Gamma \vdash e \Leftarrow \tau : (\phi_1; \Delta_1)}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{let\ mut}\ x : \tau = e;) : (\phi_1; (\Delta_1, x : \tau); \Gamma)} \quad \text{(T-Let-Mut)}$$

Array assignment

$$\frac{\begin{array}{c}(x : [\tau_{\mathsf{elem}}; N]) \in \Delta_0 \\ \phi_0; \Delta_0; \Gamma \vdash e_i \Rightarrow (\phi_1; \Delta_1; \tau_i) \quad \phi_1 \vdash \tau_i <: \mathsf{int} \\ \phi_1; \Delta_1; \Gamma \vdash e \Rightarrow (\phi_2; \Delta_2; \tau_v) \quad \phi_2 \vdash \tau_v <: \tau_{\mathsf{elem}} \\ \phi_2 \wedge \mathsf{prop}(\tau_i) \vdash 0 \leq \mathsf{val}(\tau_i) < N \quad \text{(Bounds proof)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (x[e_i] = e;) : (\phi_2; \Delta_2; \Gamma)} \quad \text{(T-Array-Assign)}$$

Assignment

$$\frac{\begin{array}{c}(x : \tau_{\mathsf{master}}) \in \Delta_0 \\ \phi_0; \Delta_0; \Gamma \vdash e \Rightarrow (\phi_1; \Delta_1; \tau_e) \\ \phi_1 \vdash \tau_e <: \tau_{\mathsf{master}}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (x = e;) : (\phi_1; \Delta_1[x \mapsto \tau_e]; \Gamma)} \quad \text{(T-Assign)}^\star$$

For loop

$$\frac{\begin{array}{c}\phi_0 \wedge P_I[e_{\mathsf{start}}/i] \quad \text{(Invariant holds on entry)} \\ \phi_0 \wedge P_I \wedge e_{\mathsf{start}} \leq i < e_{\mathsf{end}};\ \Delta_0;\ (\Gamma, i : \mathsf{int}) \vdash B : (\phi_1; \Delta_1; \Gamma) \\ \phi_1 \vdash P_I[(i+1)/i] \quad \text{(Invariant preserved by body)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{for}\ i\ \mathsf{in}\ e_{\mathsf{start}}..e_{\mathsf{end}}\ \{B\}) : (\phi_0 \wedge P_I; \Delta_0; \Gamma)} \quad \text{(T-For)}^\star$$

where P_I is the invariant proposition (or $\top$ if no invariant is specified).

Return

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e \Rightarrow (\phi_1; \Delta_1; \tau_e) \quad \phi_1 \vdash \tau_e <: \tau_{\mathsf{ret}} \\ \phi_1 \vdash P_{\mathsf{ens}}[e/\mathtt{result}] \quad \text{(Postcondition proof)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{return}\ e;) : (\phi_1; \Delta_1; \Gamma)} \quad \text{(T-Return)}^\star$$

If-statement and while loop

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e_{\mathsf{cond}} \Rightarrow (\phi_1; \Delta_1; \tau_c) \quad \phi_1 \vdash \tau_c <: \mathsf{bool} \\ \phi_1 \wedge \mathsf{prop}(\tau_c); \Delta_1; \Gamma \vdash B : (\phi_2; \Delta_2; \Gamma) \\ \phi_1 \wedge \neg\mathsf{prop}(\tau_c); \Delta_1; \Gamma \vdash B' : (\phi_3; \Delta_3; \Gamma) \\ (\phi_{\mathsf{join}}, \Delta_{\mathsf{join}}) = \mathsf{join}(\phi_2, \Delta_2, \phi_3, \Delta_3)\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{if}\ e_{\mathsf{cond}}\ B\ \mathsf{else}\ B') : (\phi_{\mathsf{join}}; \Delta_{\mathsf{join}}; \Gamma)} \quad \text{(T-If-Stmt)}$$

$$\frac{\begin{array}{c}\phi_0; \Delta_0; \Gamma \vdash e_{\mathsf{cond}} \Rightarrow (\phi_1; \Delta_1; \tau_c) \quad \phi_1 \vdash \tau_c <: \mathsf{bool} \\ \phi_0 \vdash \phi_I \quad \text{(Invariant holds on entry)} \\ \phi_I \wedge \mathsf{prop}(\tau_c);\ \Delta_0;\ \Gamma \vdash B : (\phi_2; \Delta_2; \Gamma) \\ \phi_2; \Delta_2 \vdash \phi_I \quad \text{(Invariant preserved by body)}\end{array}}{\phi_0; \Delta_0; \Gamma \vdash (\mathsf{while}\ e_{\mathsf{cond}}\ \{B\}) : (\phi_I \wedge \neg \mathsf{prop}(\tau_c); \Delta_0; \Gamma)} \quad \text{(T-While)}$$

B Complete DTAL typing rules

This appendix lists the complete set of typing rules implemented by the DTAL verifier. Each rule has the form $\Phi; \Gamma \vdash \mathsf{instr} \Rightarrow \Gamma'$, where Φ is the constraint context, Γ maps registers to types, and Γ' is the updated register map. We write $\mathsf{idx}(\tau)$ for the index expression extracted from a type (e.g., $\mathsf{idx}(\mathsf{int}(e)) = e$). Rules marked with $\star$ are also presented in Chapter 3.

Data movement

$$\frac{}{\Phi; \Gamma \vdash \mathsf{mov}\ r_d, n \Rightarrow \Gamma[r_d \mapsto \mathsf{int}(n)]} \quad \text{(D-MovImm)}^\star$$

$$\frac{\Gamma(r_s) = \tau}{\Phi; \Gamma \vdash \mathsf{mov}\ r_d, r_s \Rightarrow \Gamma[r_d \mapsto \tau]} \quad \text{(D-MovReg)}$$

$$\frac{\Gamma(r_b) = [\tau; N] \quad \Phi \vdash 0 \le \mathsf{idx}(\Gamma(r_i)) < N}{\Phi; \Gamma \vdash \mathsf{load}\ r_d, [r_b, r_i] \Rightarrow \Gamma[r_d \mapsto \tau]} \quad \text{(D-Load)}^\star$$

$$\frac{\Gamma(r_b) = [\tau; N] \quad \Phi \vdash 0 \le \mathsf{idx}(\Gamma(r_i)) < N \quad v' = v + 1 \quad \text{(fresh array version)}}{\Phi; \Gamma \vdash \mathsf{store}\ [r_b, r_i], r_s \Rightarrow \Gamma} \quad \text{(D-Store)}$$

After a store, the verifier emits two axioms into Φ: a *write axiom* ($\mathsf{select}(arr_{v'}, e_i) = e_s$) and a *frame axiom* ($\forall k.\ k \neq e_i \Rightarrow \mathsf{select}(arr_{v'}, k) = \mathsf{select}(arr_v, k)$), where v and v' are the old and new array versions.

Arithmetic

$$\frac{\Gamma(r_1) = \mathsf{int}(e_1) \quad \Gamma(r_2) = \mathsf{int}(e_2)}{\Phi; \Gamma \vdash \mathsf{binop}_\oplus\ r_d, r_1, r_2 \Rightarrow \Gamma[r_d \mapsto \mathsf{int}(e_1 \oplus e_2)]} \quad \text{(D-BinOp-Singleton)}^\star$$

$$\frac{\Gamma(r_1) = \{a : \mathsf{int} \mid \varphi_1\} \quad \Gamma(r_2) = \{b : \mathsf{int} \mid \varphi_2\}}{\Phi; \Gamma \vdash \mathsf{binop}_\oplus\ r_d, r_1, r_2 \Rightarrow \Gamma[r_d \mapsto \exists c.\ \mathsf{int}(c)\ \textbf{where}\ \exists a, b.\ \varphi_1 \wedge \varphi_2 \wedge c = a \oplus b]} \quad \text{(D-BinOp-Refined)}^\star$$

$$\frac{\Gamma(r_s) = \mathsf{int}(e)}{\Phi; \Gamma \vdash \mathsf{addimm}\ r_d, r_s, n \Rightarrow \Gamma[r_d \mapsto \mathsf{int}(e + n)]} \quad \text{(D-AddImm)}$$

$$\frac{\Gamma(r_s) = \tau \quad \tau <: \mathsf{bool}}{\Phi; \Gamma \vdash \mathsf{not}\ r_d, r_s \Rightarrow \Gamma[r_d \mapsto \mathsf{bool}]} \quad \text{(D-Not)}$$

Comparison and branching

$$\frac{\Gamma(r_1) = \tau_1 \quad \Gamma(r_2) = \tau_2}{\Phi; \Gamma \vdash \mathsf{cmp}\ r_1, r_2 \Rightarrow \Gamma \quad (\text{records operands } \mathsf{idx}(\tau_1), \mathsf{idx}(\tau_2))} \quad \text{(D-Cmp)}$$

$$\frac{\Gamma(r_1) = \tau_1}{\Phi; \Gamma \vdash \mathsf{cmp}\ r_1, n \Rightarrow \Gamma \quad (\text{records operands } \mathsf{idx}(\tau_1), n)} \quad \text{(D-CmpImm)}$$

$$\frac{}{\Phi;\Gamma \vdash \mathsf{setcc}\ r_d, \mathit{cond} \Rightarrow \Gamma[r_d \mapsto \mathsf{bool}]} \quad \text{(D-SetCC)}$$

$$\frac{\text{last cmp operands: } e_1, e_2}{\Phi;\Gamma \vdash \mathsf{branch}\ \mathit{cond}\ L \Rightarrow \Gamma \quad \text{with } \Phi' = \Phi \wedge \neg(e_1\ \mathit{cond}\ e_2)\ \text{(fall-through)}} \quad \text{(D-Branch)}^{\star}$$

The taken edge to L carries $\Phi \wedge (e_1\ \mathit{cond}\ e_2)$.

Stack and allocation

$$\frac{\Gamma(r_s) = \tau}{\Phi;\Gamma \vdash \mathsf{push}\ r_s \Rightarrow \Gamma \quad (\text{push } \tau \text{ onto typed stack})} \quad \text{(D-Push)}$$

$$\frac{S = \tau :: S'}{\Phi;\Gamma \vdash \mathsf{pop}\ r_d \Rightarrow \Gamma[r_d \mapsto \tau] \quad (\text{pop from typed stack})} \quad \text{(D-Pop)}$$

$$\frac{}{\Phi;\Gamma \vdash \mathsf{alloca}\ r_d, n \Rightarrow \Gamma[r_d \mapsto [\mathsf{int}; n]]} \quad \text{(D-Alloca)}$$

After allocation, the verifier adds a zero-initialisation axiom: $\forall k \in [0, n).\ \mathsf{select}(arr_0, k) = 0$.

Annotations

$$\frac{\Gamma(r) = \tau_{\mathsf{derived}} \quad \Phi \vdash \tau_{\mathsf{derived}} <: \tau_{\mathsf{ann}}}{\Phi;\Gamma \vdash \mathsf{type}\ r : \tau_{\mathsf{ann}}} \quad \text{(D-TypeAnn)}$$

The verifier derives the register's type independently and checks it is compatible with the compiler's annotation. If the annotation is more precise (e.g., a singleton vs. the derived existential), the verifier checks that the derived type implies the annotation under Φ.

$$\frac{\Phi \vdash C}{\Phi;\Gamma \vdash \mathsf{assert}\ C \Rightarrow \Gamma \quad (\text{add } C \text{ to } \Phi)} \quad \text{(D-Assert)}$$

The verifier checks that the constraint C is provable from the current Φ before adding it.

Function calls

$$\frac{\Sigma(f) = (\vec{x} : \vec{\tau}) \to \tau_{\mathsf{ret}}\ \mathsf{requires}\ P_{\mathsf{req}}\ \mathsf{ensures}\ P_{\mathsf{ens}} \qquad \Phi \vdash P_{\mathsf{req}}[\overline{r_i/x_i}]}{\Phi;\Gamma \vdash \mathsf{call}\ f \Rightarrow \Gamma[r_0 \mapsto \tau_{\mathsf{ret}}] \quad (\text{add } P_{\mathsf{ens}}[\overline{r_i/x_i}] \text{ to } \Phi)} \quad \text{(D-Call)}$$

The precondition is checked against Φ after substituting physical parameter registers for formal parameters. The postcondition (with `result` replaced by r_0) is assumed in Φ after the call.

Physical instructions (post-regalloc)

The following instructions appear only after register allocation and are verified for basic consistency:

- **D-Cqo**: Sign-extends `rax` into `rdx:rax`. The type of `rdx` is set to the type of `rax`.
- **D-Idiv**: Divides `rdx:rax` by the source register. Both `rax` (quotient) and `rdx` (remainder) receive type int.
- **D-SpillStore**: Records the type stored at a stack offset.

- **D-SpillLoad**: Checks that the loaded type matches what was previously stored at that offset.
- **D-Prologue / D-Epilogue**: Structural markers verified at the function level.

C Project Proposal

A type-preserving compiler from a systems language to DTAL

Athena Eng

10 October 2025

C.1 Introduction

Systems programming languages such as C and C++ offer the programmer granular control over hardware and efficient use of resources, making them appropriate for performance-critical applications. However, these languages are fundamentally memory-unsafe and can lead to security vulnerabilities that go unnoticed in production code. For example, the Heartbleed bug [2] was the result of an out-of-bounds read, which led to sensitive data being leaked.

There are several mechanisms that modern languages implement to address problems such as out-of-bounds access. Languages such as Python and Java use runtime checks and garbage collection to ensure memory safety guarantees, but these methods incur overhead that can degrade performance. The Rust programming language enforces compile-time safety using language features such as ownership and borrow-checking to prevent undefined behaviour (e.g. use-after-free and double free) that is prevalent in C and C++.

While Rust does provide a strong solution for preserving safety guarantees while maintaining performance comparable to C and C++, its correctness relies on the assumption that the compiler implementation is correct and preserves the semantics of the high-level (source) language. However, there may be situations in which a compiler for some given source language is not trusted. Projects such as CompCert [4] and CakeML [21] have developed verified compilers that ensure compiled programs behave exactly as the source program was defined.

This dissertation project aims to move the locus of trust from the compiler to a simple, independent verifier. Formal compiler verification is complex and requires a lot of work whereas a verifier can be a small and simple trusted base. Proving the full semantic equivalence between source and target programs will be beyond the scope of this project. This project will implement a toolchain that uses type-preserving compilation to guarantee safety properties and statically verify the absence of certain unsafe behaviours and classes of runtime errors. A type-preserving compiler translates well-typed source code into verifiable target code, which for this project will be a typed assembly language. The typed assembly language will use features inspired by DTAL [22], where a register's type carries information about its value (e.g. the length of an array). Future work on this project could involve developing a formally verified compiler for the target assembly language such that both correctness and safety are achieved.

C.1.1 Dependent Type Theory

Dependent type theory (DTT) is a flavour of type theory that introduces dependent types, a type whose definition depends on a value. Dependent types also feature in Martin-Löf type theory, which can serve as an alternative foundation to mathematics. The theory promotes a view of a type as a mathematical proposition, where a value of that type would be a proof of that proposition. This correspondence between programs and proofs is known as the Curry-Howard isomorphism.

Type-checking DTT with extensional equality[1] is actually undecidable. The work of Xi & Pfenning, which was inspiration for this project, implements a dependently type assembly language (DTAL) using a restricted form of DTT based on the decidable logic of linear integer arithmetic. Types are not first-class citizens as one might observe in proof languages such as Agda, but rather exist at compile-time to verify program properties. This model allows the type system to be powerful enough to express properties like loop invariants and verify them statically. Using dependent types specifically enables certain compiler optimisations that would not be possible using a more traditional type system [23]. For example, runtime bounds checks can be transformed into compile-time proofs without the overhead.

C.1.2 Safety Properties

The dependent type system of this project is intended to provide a framework for proving program properties, with a focus on proving structural properties as described in the following.

[1] extensional equality means that definitional equality is implied by propositional equality.

Spatial Memory Safety

- Guarantee that every access to a memory buffer is within its legal bounds.
- We define an array's type as a dependent pair `(int, m)`, where `m` is the array's size. Any indexing operation generates a proof obligation (e.g. `0 <= i < m`), which must be solved by the compiler frontend.

Control-Flow Integrity

- Guarantee that branch instructions only target valid destinations.
- The DTAL labels destinations with a type to represent machine state, which will be compared with the current state at a branch instruction that jumps there, which involves subtype checking.

Integer Overflows

- Guarantees that integer arithmetic operations will not overflow or underflow.
- Range/refinement types can be used to guarantee integers stay within a range. These types can also be used for refining types in select blocks e.g. `{i:int | i < 10}`).

Preconditions and Postconditions

- Guarantees that a function is only ever called with inputs that satisfy given constraints.
- The HLL will have a language feature to specify preconditions (e.g. `require(m <= n)` on a function defines a Π-type). The compiler frontend then must prove this predicate holds true at every call.

The dependent type system could also be extended to prove more complex concurrent and behavioural properties. The following properties move beyond the core project.

Data Race Freedom

- Guarantees that two threads cannot concurrently access the same memory location if at least one of those accesses is a write.
- We could use ownership and linear/affine types similar to Rust's concurrency model and send and sync types for passing data between threads.

Information Flow Control

- Guarantees that sensitive information such as private keys and passwords cannot leak into public channels (e.g. log files, network sockets).
- We could extend the type system by incorporating security labels into the type system and define a security lattice that defines legal flow of information.

Communication Protocols

- Guarantees that two threads conform to a given message-passing protocol.
- Session types can be used for defining sequences of communication over a channel by describing the types that are expected to be sent/received during communication. Types evolve to reflect the next state of the protocol.

C.2 Structure and Substance of the Project

The core of this project will involve specifying a high-level systems language as the source language, low-level target language similar to DTAL as well as intermediate languages; implementation of a compiler for compiling the systems language to DTAL; and a verifier to assert a number of properties and safety guarantees in the generated target code.

Additional features that could build on the foundation of the core project include an interpreter for the generated DTAL, extending the specified core languages and support for concurrency properties and behavioural correctness. The full list of proposed extensions for the project are listed in 4.2.

The implementation will be written in Rust, which offers safety, stability and high performance.

C.2.1 Main Components

Figure C.1 shows the components of the project and their inputs and outputs. The core components include specification of the source and target languages, the compiler and the verifier, whereas the assembler and linker for generating an object file from the DTAL will be an extension.

As stated in the introduction, the goal of this project is not to produce a formally verified compiler. Hence, in the design of this system, the compiler is untrusted, though compiler implemented in this project should perform the essential function of faithfully encoding the high-level safety specifications of the source language as explicit type annotations in the low-level code. The trusted component is the verifier, which will take the DTAL file as input and independently check that claims expressed in the type-annotated instructions logically follow from the code's execution. Hence, its correctness is paramount for providing a definite guarantee that programs are free from given classes of errors.

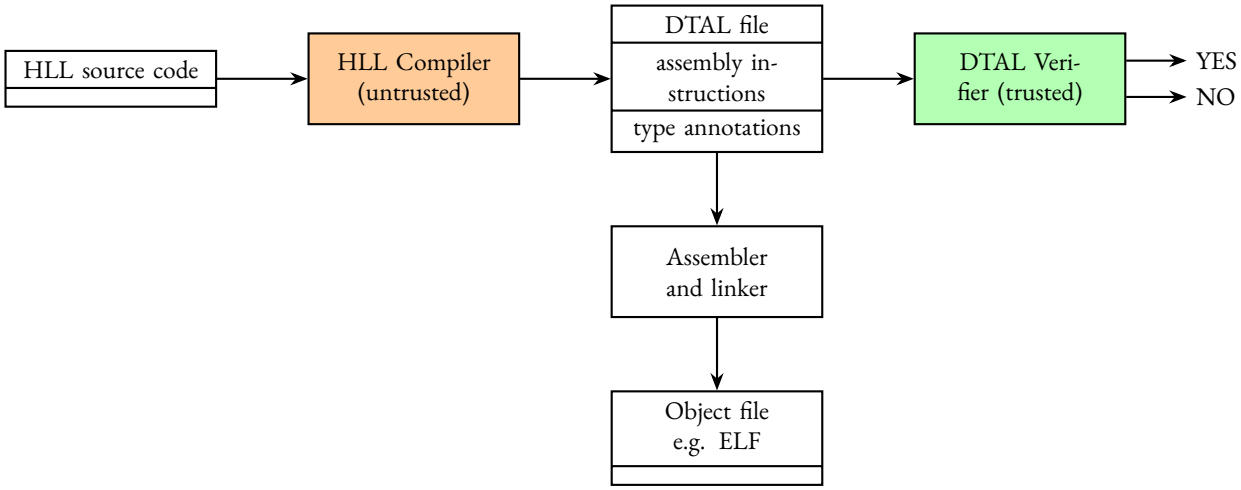

Figure C.1: System model of the project

High-level Language Design

The source language will be a minimal systems language (consisting of C-like and Rust-like features) focused on demonstrating the core safety properties.

- Constructs:
  - first-order functions (`fn`)
  - assignment (`let`)
  - if/else
  - for (`for i in 0..n`)
  - preconditions (`requires(...)`)
- Types:
  - primitive types (`int`, `bool`)
  - fixed-size array (`[T; N]`, where `T` is the element type and `N` is the size)
  - immutable reference (`&T`)
  - mutable variable (`let mut x = ...`)
  - singleton types (`int(c)`)
  - range/refinement types (`{i: int | P(i)}`, where `P` is some proposition)

DTAL Design

The target DTAL will be a RISC-like instruction set but every instruction will have a formal typing rule so it can be verified.

ISA:

- Moving data (`mov`, `load`, `store`)
- Arithmetics (`add`, `sub`, `mul`)
- Control flow (`jmp`, `cmp`, `blt`, `bgt`, `beq`, `ret`)

Type system:

- primitive types
- dependent integer types
- pointer types (`ptr(T)`)
- state types for labels (`r1:T1, ...`)
- function types for entry points (`(args...) -> ret`)

Compiler Frontend

The first stage of the compiler frontend involves lexing and parsing, which will be done with the help of a parser generator/combinator to build an abstract syntax tree (AST). There are several existing Rust crates[2] that do this. The type-checker will the main substance of work for the frontend. It will traverse the AST and apply the language's formal typing rules as well as generating and solving proof obligations. For example:

- For an array access `arr[i]`, the types of `arr` (e.g. `&(int, m)`) and `i` are synthesised and the proof obligation `i >= 0 && i < m` is generated.
- For a function with a `requires(n <= m)` clause, the function arguments are substituted into the clause to generate a proof obligation.

After the AST has been fully traversed, the frontend will have a set of logical constraints that must hold true. The constraint solving will be done with the help of an external SMT solver such as Z3. Finally, the AST will be annotated with the verified types.

Compiler Backend

The compiler backend is responsible for translating the AST from the frontend into a linear sequence of well-typed DTAL. Note that the compiler I develop will not be formally verified but it should perform proof transformations faithfully, encoding the safety guarantees verified by the frontend as correct and explicit type annotations in the target DTAL. The project's model assumes that the compiler could be incorrect, but the compiler delivered here aims to be as correct as possible by following standard engineering principles. Implementation will be semantics-driven with rigorous testing to maximise confidence of correctness.

The backend's workflow will be implemented as a multi-stage pipeline that will consist of:

- Lowering to a defined typed intermediate representation (IR) that will resemble a linear, three-address code representation structured as a CFG using simplifying assumptions e.g. (an infinite set of virtual registers).
- Instruction selection and typed code generation where each instruction in the IR is mapped to one or more DTAL instructions. This process should preserve and encode type information.
- Typed register allocation where a register allocation algorithm will be implemented that takes the type system into account.

[2] A Rust crate is a compilation unit that the Rust compiler processes at a time e.g. binaries, libraries

- Emission of DTAL file containing the finalised instructions with annotations and labels.

Some examples of transformations to DTAL include:

- Arithmetic expression translation: we can derive resulting dependent types (e.g. for `add r3, r1, r2` where `r1:int(1)` and `r2:int(2)` the result register `r3` has type `int(3)`).
- Selection statement translation: we can generate explicit labels and conditional branch instructions. Refinement types can be used to refine the type of values in registers based on conditions (e.g. `r1:int` becomes `r1:{i:int|i < 10}` for a branch where the value in r1 $<$ 10).
- For loop translation: loop invariants can be expressed as a type (e.g. `r1:{i:int|0 <= i < n}`) and we can generate loop start/end labels.

DTAL Verifier

The DTAL verifier will take the generated assembly and verify its proofs. This is the root of trust in the system so it should be small and simple (relative to the compiler). As mentioned, the compiler is large and complex whereas it is far more tractable to formally verify the verifier. This helps achieve a level of certified safety in contexts such as supply chain security or platform providers where there may zero trust in external toolchains.

The verifier will lex and parse the assembly to produce an AST for the DTAL grammar. From this, we build a control flow graph with annotated type information where each node is a basic block and the edges between blocks are the branch instructions. Using the CFG, we can execute an algorithm to perform data flow analysis that keeps track of the types of register values and prove that this is consistent with the type annotations. Some components and mechanisms used for the compiler frontend can be reused here e.g. verification of logical constraints via an SMT solver, AST traversal etc.

Within a block in the CFG, the verifier can perform actions on each instruction to check and update. For example, an instruction such as `mov r4, 0` will update the type state to express that `r4` is of the singleton type $\mathsf{int}(0)$. For an instruction `load r3, [r1, r2]` where `r1` is a pointer to an array with type $\mathsf{ptr}(\mathsf{array}(\mathsf{int}, m))$ and `r2` is an index `nat` to access the array, the algorithm would need to check `m` against proofs in the typing state to ensure that the value in `r2` is less than m.

Given that the verifier should be trusted, it will need to be rigorously tested. It is intended to be small enough such that a full manual audit would be possible. The gold standard would be formal verification via a proof assistant, which would be a major extension to the project.

C.3 Starting Point

- The Part IB Compiler Construction, Introduction to Computer Architecture and Semantics of Programming Languages courses covered concepts relevant to this project.
- I have foundational knowledge of dependent type theory, having used it as the topic of an introductory technical talk I delivered.
- I have limited practical experience with proof-carrying code and using proof assistants.
- I do not have extensive practical experience with compiler development.
- I began learning the Rust programming language over the 2025 summer vacation and worked in a Rust codebase during my internship.

C.4 Success Criterion

C.4.1 Core

- High-level language with core language features (see 2.1).
- DTAL with core language features (see 2.1). The extended version of Xi and Pfenning's paper [22] contains the full rules of a DTAL, which can be used as reference.

- Compiler frontend to fully support lexing, parsing and type-checking of the core high-level language and its type system. There are existing type-checkers for refinement type systems that can be used from reference.
- Compiler backend to fully support compilation of the core high-level language and reject unsafe programs at compile time for the core safety properties (see 1.2).
- Verifier that accepts assembly code generated from safe programs.

C.4.2 Extensions

- An interpreter and object file generator to produce an executable from the target DTAL.
- Support for modularised compilation, verification and linking.
- Widening the language and type system with more features (e.g. structs, generics, error handling, ownership, linear types).
- Prove some concurrency and behavioural correctness properties such that the verifier accepts a wider range of correct programs.

C.5 Evaluation Plan and Metrics

I will create a test suite to test the final system against safe and unsafe programs which should compile and verify, or be rejected by the DTAL verifier. With the language features defined in my core, this can write programs for performing calculations such as Fibonacci, factorial etc. and performing operations on vectors such as sorting. The Rust QuickCheck crate[3] can be used for generating programs with predefined properties to test the toolchain.

I will also measure end-to-end compilation time in order to evaluate to what extent ensuring correctness impacts performance. I will also perform a qualitative assessment of the range of safety properties that can be expressed or enforced by the language, and how this translates to its real-world application.

C.6 Timetable and Milestones

C.6.1 Michaelmas Term

- **16 Oct. - 29 Oct.**
  - Design and formally define core high-level language, DTAL and type system.
  - **Milestone**: Send formal definition of semantics and typing rules of the HLL and DTAL to supervisor for review and beginnings of their in-code implementation.
- **30 Oct. - 12 Nov.**
  - Finish with design and implementation of languages.
  - Implement lexer and parser (using parser generator/combinator).
  - **Milestone**: Send full design of languages with lexer and parser for the HLL to supervisor for review.
- **13 Nov. - 26 Nov.**
  - Revision for DSP module written test.
  - DSP module written test (21 Nov.)
  - Start implementing frontend type-checker (if time allows).
  - **Milestone**:
- **27 Nov. - 10 Dec.**

[3] https://docs.rs/quickcheck/latest/quickcheck/

  - Implement the frontend type-checker with proof obligation checking using a logical constraint solver.
  - **Milestone**: Implemented type-checker that applies formal typing rules with passing unit tests covering the major blocks of logic; compiler frontend implementation complete and ready for supervisor review.

C.6.2 Michaelmas Vacation

- **11 Dec. - 24 Dec.**
  - Break
  - Design IR and implement translation of HLL to IR.
  - **Milestones**: Complete design of IR and translator.
- **25 Dec. - 7 Jan.**
  - Implement translation of IR to DTAL.
  - **Milestones**: Complete translator that lowers IR to DTAL and preserves type information.
- **8 Jan. - 21 Jan.**
  - Implement register allocation algorithm.
  - Implement emitter to output final DTAL code.
  - **Milestone**: Majority of compiler backend implementation complete.

C.6.3 Lent Term

- **22 Jan. - 4 Feb.**
  - Progress report and presentation.
  - Start implementing verifier.
  - **Milestones**: Delivered progress report and presentation and have beginnings of a verifier implementation (lexer, parser, CFG construction).
- **5 Feb. - 18 Feb.**
  - Plan structure of dissertation.
  - Finish verifier implementation (data flow analysis logic)
  - **Milestone**: Verifier implementation complete.
- **19 Feb. - 3 Mar.**
  - Write dissertation introduction.
  - Begin end-to-end testing and evaluation.
  - **Milestone**: Submit draft of dissertation to supervisor and obtain some evaluation results.
- **4 Mar. - 17 Mar.**
  - Write dissertation preparation section.
  - Buffer time.
  - **Milestone**: Submit draft of dissertation to supervisor.

C.6.4 Easter Vacation

- **18 Mar. - 31 Mar.**
  - Write dissertation implementation section.
  - Extensions
  - **Milestone**: Submit draft of dissertation to supervisor.
- **1 Apr. - 14 Apr.**

  - Write dissertation evaluation section.
  - Extensions
  - **Milestone**: Submit full draft of dissertation to supervisor.

- **15 Apr. - 28 Apr.**
  - Work on drafts of dissertations.
  - Extensions
  - **Milestone**: Dissertation approved by supervisor.

C.6.5 Easter Term

- **29 Apr. - 10 May**
  - Final checks.
  - **Milestone**: Dissertation ready for submission.

C.7 Resources Declaration

I will be developing on my laptop for this project. It has a 13th Gen Intel(R) Core(TM) i7-1365U processor and 32 GB RAM. I will make regular backups of the code to a private GitHub repository and a USB drive.

Bibliography

[1] Leslie Lamport. Proving the correctness of multiprocess programs. *IEEE Transactions on Software Engineering*, SE-3(2):125–143, 1977.

[2] The MITRE Corporation. CVE-2014-0160. Common Vulnerabilities and Exposures, April 2014. URL: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160.

[3] Nancy G. Leveson and Clark Savage Turner. An investigation of the therac-25 accidents. *Computer*, 26:18–41, 1993.

[4] Xavier Leroy. Formal verification of a realistic compiler. *Commun. ACM*, 52(7):107–115, July 2009.

[5] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 106–119. ACM, 1997.

[6] Patrick M. Rondon, Ming Kawaguchi, and Ranjit Jhala. Liquid types. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 159–169. ACM, 2008.

[7] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Proceedings of TACAS 2008*, volume 4963 of *LNCS*, pages 337–340. Springer, 2008.

[8] Hongwei Xi and Robert Harper. A dependently typed assembly language. In *Proceedings of the 6th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 169–180. ACM, 2001.

[9] Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375, 1978.

[10] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.

[11] Hongwei Xi and Frank Pfenning. Dependent types in practical programming. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 214–227. ACM, 1999.

[12] Benjamin C. Pierce and David N. Turner. Local type inference. In *ACM Transactions on Programming Languages and Systems*, volume 22, pages 1–44, 2000.

[13] Clark Barrett, Morgan Deters, Leonardo de Moura, Albert Oliveras, and Aaron Stump. 6 years of SMT-COMP. *Journal of Automated Reasoning*, 50(3):243–277, 2013.

[14] Greg Morrisett, David Walker, Karl Crary, and Neal Glew. From System F to typed assembly language. volume 21, pages 527–568. ACM, 1999.

[15] Hongwei Xi. Imperative programming with dependent types. In *15th Annual IEEE Symposium on Logic in Computer Science, Santa Barbara, California, USA, June 26-29, 2000*, pages 375–387. IEEE Computer Society, 2000.

[16] Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton Jones. Refinement types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 269–282. ACM, 2014.

[17] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Proceedings of LPAR-16*, volume 6355 of *LNCS*, pages 348–370. Springer, 2010.

[18] Nikhil Swamy, Cătălin Hriţcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean-Karim Zinzindohoue, and Santiago Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 256–270. ACM, 2016.

[19] Michael Sammler, Rodolphe Lepigre, Robbert Krebbers, Kayvan Memarian, Derek Dreyer, and Deepak Garg. RefinedC: Automating the foundational verification of C code with refined ownership types. In *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 158–174. ACM, 2021.

[20] Hongwei Xi. Applied type system (extended abstract). In *Proceedings of the TYPES Workshop*, volume 3085 of *LNCS*, pages 394–408. Springer, 2003.

[21] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. CakeML: A verified implementation of ML. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 179–191. ACM, 2014.

[22] Hongwei Xi and Robert Harper. A dependently typed assembly language. *Technical Report CMU-CS-01-169, Carnegie Mellon University*, 2001. Extended version of the ICFP 2001 paper.

[23] Greg Morrisett, David Walker, Karl Crary, and Neal Glew. From System F to typed assembly language. In *Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 85–97. ACM, 1998.